

StaffML: A Physics-Grounded Interview Question Bank for Machine Learning Systems Engineers

System Design, Competency Model, and Quality Infrastructure

Vijay Janapa Reddi
Harvard University

staffml.ai

Abstract

As large language models commoditize algorithmic coding, the ability to reason quantitatively about physical hardware constraints, a skill known as *mechanical sympathy*, has become the defining skill for Staff-level ML systems engineers, yet no structured assessment resource exists for it. We introduce the *ikigai competency model*, which decomposes this expertise into four fundamental skills (recall, analyze, design, quantify) whose pairwise intersections yield 11 compound cognitive zones (diagnosis, specification, fluency, and others) that capture qualitatively different acts of engineering reasoning. Building on this model, we present STAFFML, a bank of 9,199 interview questions classified on four independent axes: *topic* (87 concepts with 123 prerequisite edges), *zone* (cognitive mode), *track* (physics regime across cloud, edge, mobile, and TinyML), and *level* (Bloom’s depth mapped to industry ladders). Every question requires napkin math grounded in real hardware specifications; a physics-based applicability matrix excludes 83 topic-track pairs where concepts lack physical substrate. Corpus integrity is enforced by a 19-check invariant system, multi-model math verification, and three-stage deduplication, all generated from a single LinkML schema. STAFFML turns Staff-level systems reasoning into something candidates can practice, hiring teams can compose interview loops from, and educators can map curricula against.

1 Introduction

The ML systems engineer sits at the intersection of hardware and software. Unlike ML researchers who

design model architectures or software engineers who write application code, ML systems engineers reason about the physical constraints of deploying machine learning at scale: memory bandwidth limits that determine whether a workload is compute-bound or memory-bound, thermal envelopes that cap sustained throughput on edge devices, network congestion that dominates training time in distributed clusters, and the economics of accelerator fleets where a single misconfigured batch size can waste thousands of dollars per hour.

This is what Martin Thompson calls *mechanical sympathy* (Thompson, 2011), a term borrowed from racing driver Jackie Stewart’s philosophy that a driver performs best when they understand what the car is doing underneath them. A Formula 1 driver who understands the engine does not merely drive faster; they drive differently, making decisions that are physically impossible to arrive at through pure algorithmic thinking. Similarly, an ML systems engineer who understands that autoregressive decode reads the full 140 GB model weights for every single token, achieving an arithmetic intensity of just 1 FLOP/Byte, far below the hardware’s 148 FLOP/Byte ridge point, does not merely optimize code; they architect systems that batch requests to amortize the bandwidth cost.¹

As large language models commoditize algorithmic coding (OpenAI, 2023), the traditional coding interview has lost its predictive power for Staff+ hiring. You cannot prompt your way out of a memory wall. What remains defensible is mechanical sympathy:

Part of the *Machine Learning Systems* book ecosystem; book context: <https://mlsysbook.ai/staffml/>.

¹Throughout this paper, *Staff+* follows common industry shorthand for Staff-and-above individual-contributor levels (e.g., Senior Staff, Principal). We use *Staff-level* for the same senior band when we discuss skills, roles, or interviews.

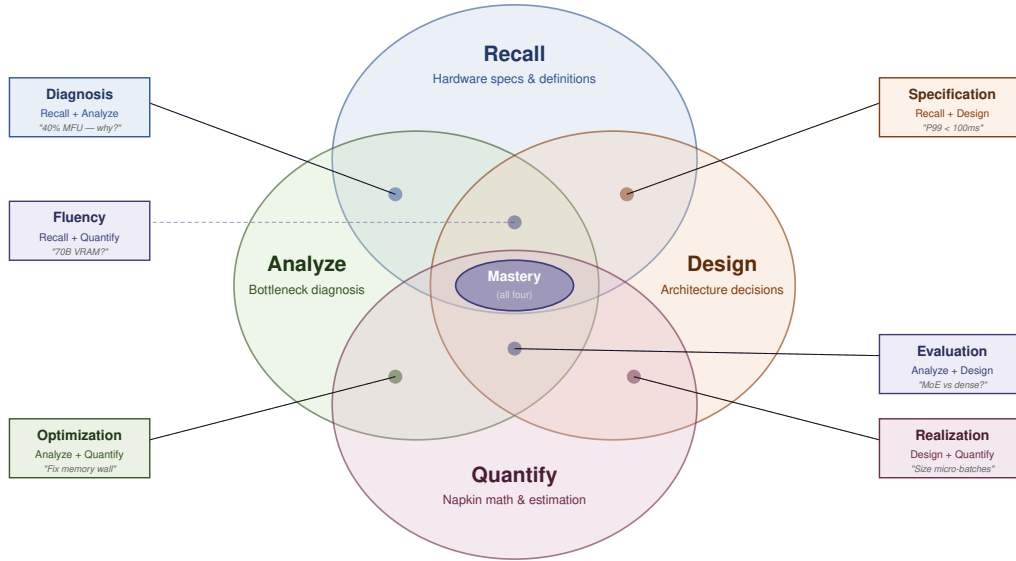


Figure 1. The STAFFML ikigai competency model. Four fundamental skills (Recall, Analyze, Design, Quantify) overlap to produce six compound zones and one central mastery zone. Each question targets exactly one zone, which determines how a topic is assessed. The compound zones capture the cognitive acts that distinguish Staff-level engineers: diagnosing failures ($\text{Recall} \cap \text{Analyze}$), translating specifications into architectures ($\text{Recall} \cap \text{Design}$), performing napkin math from memory ($\text{Recall} \cap \text{Quantify}$), evaluating architectural alternatives ($\text{Analyze} \cap \text{Design}$), sizing concrete systems ($\text{Design} \cap \text{Quantify}$), and optimizing bottlenecks ($\text{Analyze} \cap \text{Quantify}$).

quantitative reasoning grounded in real hardware specifications.²

Yet interview preparation resources for this role are thin. LeetCode ([LeetCode Inc., 2024](#)) offers 2,800+ algorithmic coding problems but nothing on GPU memory hierarchies or KV-cache sizing. Huyen’s *Designing Machine Learning Systems* ([Huyen, 2022](#)) provides strong conceptual coverage but no quantitative practice (see Section 13 for a detailed comparison). Mock interview platforms focus on behavioral and coding rounds, not system design with napkin math.³ The result: candidates study the wrong material, and interviewers lack a validated question bank calibrated to the cognitive demands of systems reasoning.

STAFFML rests on one thesis: **constraints drive architecture**. You do not choose a Transformer because it is trendy; you choose it because its attention

mechanism parallelizes across thousands of cores on real silicon, unlike the sequential dependency chains of RNNs. You do not use mixed precision because a tutorial told you to; you use it because FP16 tensor cores deliver $2\times$ the throughput of FP32 and halve the memory footprint, changing whether your model fits on one GPU or requires four. This philosophy, drawn from the *Machine Learning Systems* textbook ([Reddi, 2026b](#)), insists that AI is infrastructure, and infrastructure has laws.

Five laws govern ML systems architecture:

1. **The Roofline Bound** ([Williams et al., 2009](#)). Every workload is either compute-bound or memory-bandwidth-bound, determined by its arithmetic intensity relative to the hardware’s ridge point.
2. **The Memory Capacity Wall**. Model weights, optimizer state, activations, and KV-cache must fit simultaneously in available memory; exceeding capacity forces parallelism, offloading, or compression.
3. **The Communication Tax**. In distributed systems, gradient synchronization, tensor transfers, and pipeline bubbles consume time proportional to model size and inversely proportional to interconnect bandwidth.

²We use *physics-grounded* to mean that every question has a numerical answer derivable from real hardware specifications. The companion MLSysIM framework ([Reddi, 2026c](#)) provides the shared constants from which all hardware numbers are drawn.

³*Napkin math* (back-of-envelope calculation) is rapid quantitative estimation from first principles. For example: “How much VRAM does Llama-70B’s KV-cache need at 128K context?” $\rightarrow 2 \times 80 \times 8 \times 128 \times 128K \times 2 \approx 42 \text{ GB}$. The candidate derives the answer in real time from five numbers: layers, KV-heads, head dimension, sequence length, and bytes per element.

4. **The Power Envelope.** Sustained throughput is bounded by thermal dissipation, not peak compute; real performance is the minimum of compute capability and cooling capacity.
5. **The Cost Multiplier.** Every architecture decision compounds through fleet size, electricity cost, and amortization period; a $2\times$ efficiency improvement at 1,000 GPUs saves millions of dollars annually.

Every STAFFML question ultimately tests reasoning about one or more of these laws. The topic taxonomy maps concepts to the laws they exercise; the zone axis determines how that reasoning is assessed.

STAFFML fills this gap with thousands of questions that test mechanical sympathy across the full ML deployment spectrum. Every question contains a realistic interview scenario drawn from industry practice, a common mistake candidates make, a solution with quantitative reasoning, napkin math grounded in real hardware specifications, and classification by topic (what concept), zone (how it is tested), track (where it deploys), and level (how hard it is). STAFFML is the companion assessment tool for the *Machine Learning Systems* textbook (Reddi, 2026b), developed alongside the Harvard CS249r course (Reddi, 2024).

Here is an example of what a question testing the Power Envelope (Law 4) looks like:

duty-cycling
fluency • tinyml • L3

Scenario: Your wildlife camera classifies birds on a Cortex-M4, waking once per second for a 20 ms inference at 50 mW active power. Sleep power is 10 μ W. Powered by two AA batteries (3,000 mAh at 3 V), how long will the device last?

Napkin math: Average power at 2% duty cycle:

$$\underbrace{0.02}_{\text{duty}} \times \underbrace{50}_{\text{mW}} + \underbrace{0.98}_{\text{sleep}} \times \underbrace{0.01}_{\text{mW}} \approx \mathbf{1.01 \text{ mW}}$$

Battery: $9,000 \text{ mWh} \div 1.01 \text{ mW} \approx 8,900 \text{ hours} \approx \mathbf{1 \text{ year}}$.

Answer: At 2% duty cycle, two AA batteries sustain a year of continuous monitoring. The constraint is sleep current, not inference compute. The power envelope at TinyML scale is a battery budget, not a thermal limit. Section 11 presents derivations across four cognitive zones.

To structure this assessment, Figure 1 summarizes the *ikigai* competency model: four fundamental skills overlap into six compound zones and a central mastery region. Section 3 defines each zone and relates them to Bloom’s levels.

Three ways to use STAFFML. STAFFML serves three audiences:

1. **Interview preparation.** Candidates filter questions by target role level (L4–L6+), practice zone, and deployment track. Depth chains provide structured study paths from fundamentals to synthesis (Section 3.5).
2. **Interview design.** Hiring managers select questions across zones to construct balanced interview loops. The zone model supplies a shared vocabulary for debrief (for example, “strong in diagnosis, weak in specification”), described further in Section 12.
3. **Curriculum alignment.** Educators map course content to the topic taxonomy to see which competency cells their curriculum covers and where gaps remain; the prerequisite graph in Section 4 suggests teaching order.

Road map. Section 2 explains how the assessment space is derived before items are written. The following sections develop the four-axis competency model, taxonomy and deployment tracks, item construction and quality infrastructure, and corpus coverage; Sections 11 and 12 illustrate questions and hiring-loop design. Section 13 situates STAFFML against prior resources; Section 14 presents our limitations and future work.

2 Methodology: Backward Design

The structure of STAFFML is a derivation, not an organizational convenience. We follow the *backward design* methodology of Wiggins and McTighe (Wiggins and McTighe, 2005), which proceeds in three stages: identify the desired results, determine acceptable evidence, and only then design the assessment. The corpus size is an output of this process, not an input. We defined the competency space, filtered it by physics, bounded its capacity empirically, and let the total fall out. Figure 2 illustrates the five-step derivation chain. Each step constrains the next, and the final corpus is fully determined by the intersection of those constraints.

2.1 The Derivation Chain

Step 1. Decompose competency into skills and zones. We start from the desired result: what must a Staff-level ML systems engineer demonstrate? Mechanical sympathy, the ability to connect hardware constraints to architectural decisions through quantitative reasoning. We decompose this into four

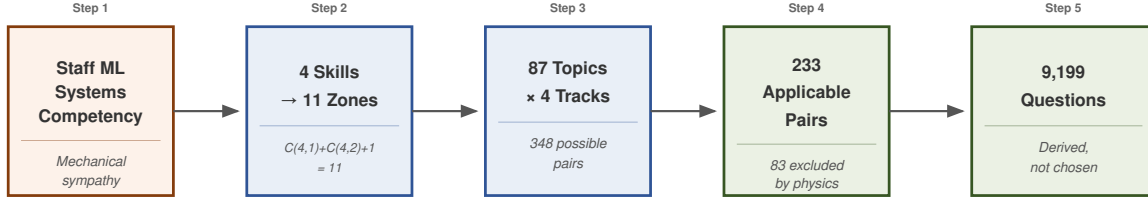


Figure 2. The backward design derivation chain. Corpus size is an output of principled constraints, not an input. Each step narrows the assessment space: competency decomposition yields 11 zones, topic curation yields 87 concepts, the physics filter excludes impossible hardware pairings, and capacity analysis bounds meaningful question density.

fundamental skills (recall, analyze, design, quantify) and observe that real engineering tasks never exercise one skill in isolation. A diagnosis requires recall and analysis; napkin math requires recall and quantification. The pairwise combinations of four skills yield six compound zones, plus one mastery zone that exercises all four. This produces the 11 cognitive zones described in Section 3.

The zone model complements Bloom’s revised taxonomy (Anderson et al., 2001). Bloom’s two-dimensional framework (cognitive process $\times$ knowledge type) captures the depth of cognitive engagement but not the simultaneous exercise of multiple cognitive modes that characterizes real engineering tasks. Consider fluency (recall $\cap$ quantify): an engineer who retrieves hardware specifications from memory and executes napkin math in real time. Bloom has no single level for this act. The *ikigai* model treats skills as dimensions, not levels. The zones are the cells of a combinatorial lattice: $\binom{4}{1} + \binom{4}{2} + 1 = 11$.

Step 2. Define topic coverage. We identify 87 topics spanning 13 competency areas as the minimum spanning set of Staff-level ML systems knowledge. The prerequisite graph (123 edges) encodes dependencies.

Step 3. Apply the physics filter. Not every topic has a physical substrate on every hardware tier. We derive an *applicability matrix* that excludes 83 topic–track pairs with one-sentence physics justifications. The remaining 233 pairs define the valid assessment space. This establishes *content validity* (Messick, 1995): every question references a concept with

physical meaning on the targeted hardware tier.

Step 4. Bound capacity empirically. For each valid cell, how many meaningfully distinct questions exist? Capacity varies by cognitive complexity and difficulty level. We treat this as an empirical question: the published corpus and coverage statistics (Section 10) make density visible without fixing a prior quota per cell.

Step 5. Lock structure before scaling. The taxonomy, applicability matrix, and competency tags are treated as part of the assessment specification: they are versioned with the schema so every item maps to a defensible coordinate in the assessment space.

Content validity is structurally enforced by the pipeline; construct validity, whether zones capture psychometrically distinct competencies, remains future work (Section 14) pending candidate response data. The next section details the derivation chain’s foundational step: the *ikigai* competency model.

3 The StaffML Competency Model

What makes ML systems expertise different from algorithmic programming skill? A software engineer who can implement a Transformer from scratch (Reddi, 2026d) may still be unable to answer a deceptively simple question: “How much VRAM does that model need, and why does that number determine your entire serving architecture?” The gap is not knowledge of algorithms; it is mechanical sympathy: the ability to reason quantitatively about how software interacts with physical hardware.

The TinyTorch competency matrix (Reddi, 2026d) defines what to teach through 8 knowledge areas crossed with 5 capability levels; the STAFFML *ikigai*

model defines how to assess whether those competencies have been acquired.

ML systems expertise is not a single skill but the intersection of multiple cognitive modes. An engineer who recalls every hardware specification but cannot analyze a failure is incomplete. An engineer who designs elegant architectures but cannot produce napkin math to validate them is dangerous. The competency model must capture these intersections.

3.1 Four Fundamental Skills

We decompose ML systems expertise into four skills. The model is inspired by *ikigai*⁴ (the intersection of overlapping life domains), adapted here as four skill domains whose intersections define qualitatively different cognitive acts:

1. **Recall.** Facts, definitions, hardware specifications. “What is the HBM bandwidth of an H100?” The engineer retrieves a stored constant.
2. **Analyze.** Tradeoffs, reasoning, root cause analysis. “Why is this workload memory-bound?” The engineer reasons about the interaction between compute and data movement.
3. **Design.** Architecture decisions, requirements-to-system mapping. “Design a serving stack with $P99 < 100\text{ms}$.” The engineer translates constraints into an architecture.
4. **Quantify.** Napkin math & estimation. “Calculate the VRAM required for a 70B model at 128K context.” The engineer produces a specific number from a formula and hardware specs.⁵

Figure 1 renders this list geometrically: each labeled overlap is a compound assessment mode (two skills at once), and the center is mastery (all four).

These four skills are not Bloom’s levels; they are orthogonal dimensions of expertise. A senior engineer may have deep recall and strong quantification skills (fluency) but weak design ability; another may excel at analysis and design (evaluation) but struggle with napkin math. The *ikigai* model makes these profiles visible and assessable.

⁴*Ikigai* is a Japanese concept describing the intersection of passion, skill, need, and reward. We adapt its four-circle Venn diagram structure to model intersecting cognitive skills in ML systems engineering.

⁵We use “quantify” rather than “implement” to emphasize that this skill tests the ability to produce concrete numbers from specifications, not the ability to write production code. The distinction matters: quantification is a reasoning act (estimating, bounding, computing), whereas implementation implies building artifacts.

Table 1. Zone definitions. The 11 *ikigai* zones with constituent skills and descriptions.

Zone	Skills	Description
Recall	recall	Retrieve facts, definitions, and specifications.
Analyze	analyze	Reason about system behavior and tradeoffs.
Design	design	Make architecture decisions and map requirements.
Quantify	quantify	Produce numbers from formulas and constants.
Diagnosis	recall + analyze	Identify root causes from observable symptoms.
Specification	recall + design	Translate constraints into an architecture.
Fluency	recall + quantify	Perform napkin math from memory.
Evaluation	analyze + design	Select the best architecture for constraints.
Realization	design + quantify	Turn decisions into concrete numbers.
Optimization	analyze + quantify	Calculate the impact of each optimization step.
Mastery	all four	Full Staff-level synthesis requiring all skills.

3.2 Eleven Cognitive Zones

The four skills combine into 11 zones: 4 pure zones (exercising a single skill), 6 compound zones (exercising two skills simultaneously), and 1 mastery zone (all four). Table 1 lists each zone with its constituent skills and a concise description; Table 2 gives typical mastery-level bands per zone (used as an authoring prior, not a separate *ikigai* axis). We deliberately omit the four possible three-skill intersections (e.g., $\text{recall} \cap \text{analyze} \cap \text{design}$) because, in practice, questions exercising three skills almost always exercise all four, making them mastery questions. The two-skill compound zones represent the cognitively distinctive acts; adding three-skill zones would dilute rather than sharpen the classification.

Pure zones test a single skill in isolation:

- **Recall.** “What is the ridge point of an H100?” Retrieve a fact.
- **Analyze.** “Why does GPU utilization collapse at batch size 1?” Reason about system behavior.
- **Design.** “Architect a KV-cache paging system.” Produce an architecture.
- **Quantify.** “Write the formula for KV-cache memory.” Produce a concrete artifact.

Compound zones test two skills simultaneously. These are the zones that most sharply differentiate strong from weak candidates, because they require integrating knowledge across cognitive modes:

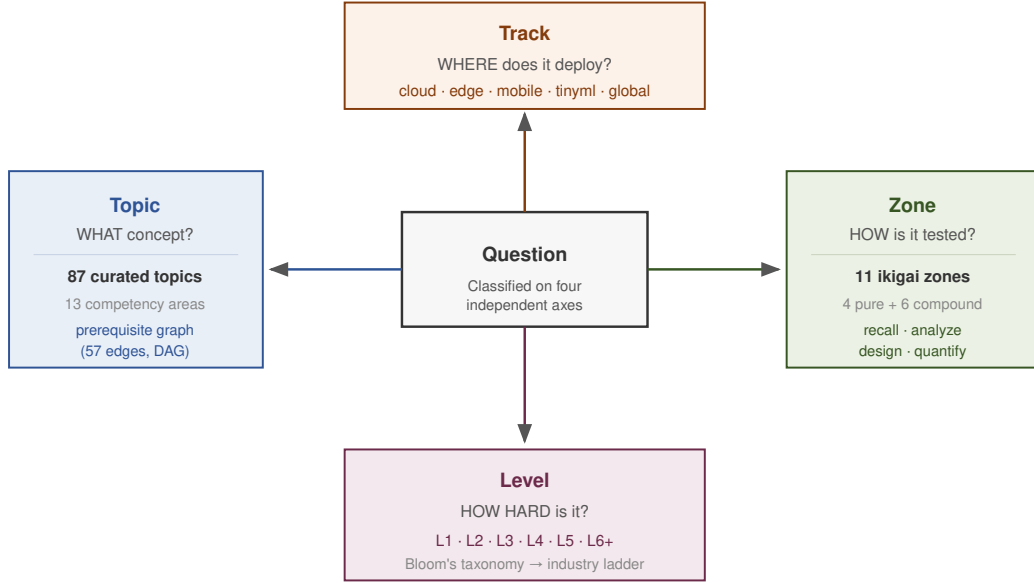


Figure 3. Four-axis classification. Every question is independently classified by *topic* (what concept is tested), *zone* (how the concept is tested), *track* (which physics regime applies), and *level* (what cognitive depth is required). The axes are orthogonal: any valid combination defines a unique assessment cell.

Table 2. Zone–level affinity. Pure zones tend toward lower Bloom’s levels; compound zones span mid-to-senior; mastery is Staff+.

Zone	Skills	Typ. levels
Recall	recall	L1–L2
Quantify	quantify	L2–L3
Fluency	recall + quantify	L2–L3
Analyze	analyze	L3–L4
Diagnosis	recall + analyze	L3–L4
Design	design	L4–L5
Specification	recall + design	L4–L5
Optimization	analyze + quantify	L4–L5
Evaluation	analyze + design	L5–L6+
Realization	design + quantify	L5–L6+
Mastery	all four	L6+

- **Diagnosis** (Recall $\cap$ Analyze). “MFU is 40%, diagnose it.” The candidate must recall hardware specifications and reason about which bottleneck explains the gap. This is the “3 AM on-call” skill.
- **Specification** (Recall $\cap$ Design). “P99 < 100 ms with 70B model, design it.” The candidate must recall hardware capabilities and translate the latency constraint into an architecture (tensor parallelism degree, batch size, KV-cache budget).
- **Fluency** (Recall $\cap$ Quantify). “How much VRAM does a 70B model need?” The candidate must recall the formula and hardware specs, then execute the arithmetic. This is napkin math

from memory, the signature skill of Staff+ engineers.

- **Evaluation** (Analyze $\cap$ Design). “MoE vs. dense: which architecture for this serving workload?” The candidate must analyze the tradeoffs and reason about which design better satisfies the constraints. No recall is required; the specs are given.
- **Realization** (Design $\cap$ Quantify). “Size the micro-batches for this pipeline-parallel training job.” The candidate must design the pipeline schedule and produce concrete numbers (batch splits, memory per stage, bubble fraction).
- **Optimization** (Analyze $\cap$ Quantify). “Fix the memory wall.” The candidate must analyze the bottleneck and produce a concrete fix (kernel fusion, quantization, offloading) with measurable improvement.

Mastery (all four skills) represents full Staff-level synthesis: “Your LLM serving cluster’s TTFT spiked from 100 ms to 4 s after a traffic surge. Diagnose the root cause, design a fix, size the required resources, and estimate the cost.” This zone requires recalling hardware specs, analyzing the failure mode, designing an architectural solution, and executing the napkin math to validate it.

3.3 Zone-Level Affinity

Each zone has a natural affinity with certain mastery levels, though the mapping is not rigid. Table 2 summarizes typical mastery-level ranges per zone. This is not a fifth “axis” layered on top of the ikigai model; it is a soft constraint used during authoring (see below). The diagram that places level alongside topic, zone, and track appears in the next subsection (Figure 3), where we define the operational four-axis framework.

This affinity is enforced as a soft constraint during question generation. An L1 question tagged as “evaluation” is flagged for review, since evaluation (analyze $\cap$ design) is cognitively inconsistent with Bloom’s “Remember” level.

3.4 The Four-Axis Classification

The competency model answers what cognitive skills matter. To operationalize this into a practical assessment framework, STAFFML classifies every question on four independent axes, each capturing a distinct dimension of ML systems expertise. Figure 3 is the road map for those four stored coordinates (topic, zone, track, level). It is intentionally separate from Table 2: the table only summarizes which difficulty bands pair naturally with each zone; the figure names every axis the corpus records, including how level fits beside topic, zone, and track.

The **topic axis** identifies which concept a question tests. The 87 topics are organized into 13 competency areas, from memory hierarchies and parallelism strategies to power budgeting and reliability, and connected by 123 typed edges (prerequisite, broader, narrower, related) that encode the dependency structure of ML systems knowledge.

The **zone axis** captures how a concept is tested, using the 11 ikigai zones defined above. It determines the cognitive profile of a question independently of its subject matter.

The **track axis** specifies where the scenario takes place, which physics regime governs the problem. Cloud questions operate at PFLOPS scale under memory bandwidth and network fabric constraints. Edge questions operate under thermal envelopes and real-time deadlines. Mobile questions face battery budgets and shared-resource contention. TinyML questions confront SRAM limits and hard real-time on bare metal. A fifth “global” track captures cross-cutting principles.

The fourth axis, **Level**, encodes cognitive depth mapped to Bloom’s taxonomy (Anderson et al., 2001):

L1 (Remember) through L6+ (Create). The levels also correspond to industry engineering ladders from intern through Staff+.

These four axes are orthogonal by design. A question about KV-cache management (topic) can test fluency (zone) in the cloud regime (track) at L3 difficulty (level), or it can test diagnosis in the edge regime at L4. The three-dimensional coverage space of topic $\times$ zone $\times$ track contains $87 \times 11 \times 4 = 3,828$ cells (excluding the cross-cutting global track). While Level acts as an independent coordinate, it exhibits a strong natural affinity with the Zone axis (e.g., Recall questions skew L1-L2, Mastery skews L6+), which we enforce as an authoring prior. Restricting to valid combinations via the applicability matrix (Section 10) yields 2,563 meaningful cells. The combinatorial structure enables precise gap analysis: the coverage space reveals exactly which assessment cells are populated, underfilled, or empty.

3.5 Depth Chains

Questions on the same topic are linked into *depth chains* that progress through Bloom’s levels, testing increasingly sophisticated cognitive acts. A chain ensures that assessment covers the full range from recall to synthesis. The corpus contains 979 chains spanning multiple levels.

Each step in the chain builds on the previous, and the zone shifts from recall through quantification to diagnosis and finally mastery. Chains serve three purposes: (1) validating that the topic taxonomy supports questions at multiple cognitive levels, (2) enabling progressive study paths where candidates build from fundamentals to synthesis, and (3) providing interviewers with calibrated question sequences that reveal the candidate’s ceiling.

For example, a chain on KV-cache management progresses through six levels and six zones:

- L1 (Recall):** “What is a KV-cache and why does it grow with sequence length?”
- L2 (Quantify):** “Calculate the KV-cache size for Llama-70B at 128K context.”
- L3 (Fluency):** “Your 80 GB GPU OOMs at 128K context. Derive the memory breakdown.”
- L4 (Diagnosis):** “PagedAttention reduces memory by 60%. Explain the mechanism.”
- L5 (Specification):** “Design a KV-cache compression strategy. What precision trade-offs are acceptable?”
- L6+ (Mastery):** “Your paged KV-cache suffers fragmentation under variable-length batching. Diagnose, design a fix, and size the memory overhead.”

The corpus contains 979 topic chains. Of these, 83 span 2 levels, 642 span 3 levels, 115 span 4 levels, 82

Table 3. Competency areas. The 13 areas containing 87 curated topics, organized by systems domain.

Area	Scope	Topics
Architecture	System design patterns	7
Compute	Accelerators, execution models	6
Cross-cutting	Compound AI, MLOps, responsible AI	5
Data	Pipelines, storage, preprocessing	7
Deployment	Serving, compression, inference	7
Latency	Scheduling, prefill/decode, SLOs	6
Memory	VRAM, KV-cache, offloading	8
Networking	Collective ops, interconnects, fabric	6
Optimization	Kernel fusion, graph rewrites, profiling	7
Parallelism	Data/tensor/pipeline/3D parallelism	6
Power	Thermal, energy, battery, DVFS	5
Precision	Quantization, mixed precision, formats	3
Reliability	Fault tolerance, checkpointing, recovery	6
Total		87

span 5 levels, and 57 span 6 levels; 23 are complete chains with at least one question at every level from L1 to L6+. In total, 3,115 questions (33.9% of the corpus) participate in at least one chain.

4 Topic Taxonomy

The taxonomy defines what each question tests, organizing ML systems engineering into a structured knowledge graph. This section describes the curated topics, their prerequisite relationships, and the knowledge organization methodology that governs selection and maintenance.

4.1 Curated Topics

The STAFFML taxonomy contains 87 topics organized into 13 competency areas (Table 3). Topics must satisfy three inclusion filters:

1. **Systems relevance.** Does the concept have measurable systems implications (compute, memory, latency, throughput, cost, or power)? If you cannot write a napkin-math question about it, it does not belong.
2. **Interview currency.** Would a hiring committee at a top-5 AI lab expect a Staff ML systems engineer to reason about this concept within the next two years?
3. **Reasoning depth.** Can questions be asked at 3+ Bloom’s levels? A concept that supports only L1 recall is too shallow.

Table 4 lists all 87 topics grouped by competency.

4.2 The Prerequisite Graph

Topics are connected by 123 typed edges following the SKOS vocabulary (W3C, 2009):

- **Prerequisite** (57 edges). Topic A must be understood before Topic B. Example: `gpu-compute-architecture` requires `roofline-analysis` as a prerequisite, because diagnosing whether a GPU kernel is compute-bound or memory-bound requires the roofline model. The graph is verified to be acyclic with a maximum prerequisite depth of 3.
- **Broader/Narrower** (14 edges). Hierarchical containment. Example: `parallelism-strategies` is broader than `tensor-parallelism`.
- **Related** (52 edges). Lateral connection. Example: `roofline-analysis` is related to `profiling-bottleneck-analysis`.

The prerequisite edges enable automated learning path generation. Given a target topic, the system computes the transitive closure of prerequisites and produces an ordered study sequence. The graph has 31 root topics (concepts with no prerequisites) and a maximum prerequisite depth of 3, ensuring that no learning path requires more than three preparatory topics. Figure 4 illustrates a subset of this graph for the parallelism area.

To show how the prerequisite graph guides study, consider a candidate targeting distributed training expertise. The graph suggests the following path through five topics spanning three competency areas:

```
roofline-analysis → gpu-compute-arch. →
data-parallelism → collective-comm. →
3d-parallelism
```

At each node, the candidate starts with L1–L2 recall and fluency questions to build foundational knowledge, then progresses to L3–L4 diagnosis and evaluation questions before moving to the next topic. The full path from roofline fundamentals to 3D parallelism mastery spans approximately 60 questions across 20 depth chains.

The prerequisite graph currently caps depth at 3. Real ML systems knowledge can require longer chains. For example, understanding why a specific collective communication pattern matters requires understanding network topology, bandwidth-latency tradeoffs, and the roofline model, a chain of depth 4 or more. Deepening the prerequisite graph is planned for future taxonomy revisions, as more topics cross the inclusion threshold and longer chains become expressible.

Table 4. Full topic taxonomy. All 87 curated topics organized by competency area.

Area	Topic Name	Area	Topic Name	Area	Topic Name
Architecture	Model Architecture Selection	Data	Data Pipeline Engineering	Networking	Collective Operations
Architecture	Model Serving Infrastructure	Data	Data Quality & Validation	Networking	Interconnect Topology
Architecture	Attention Mechanism Systems	Data	Feature Store Systems	Networking	Network Bandwidth Modeling
Architecture	Mixture-of-Experts Systems	Data	Data Storage & Formats	Networking	RDMA & InfiniBand
Architecture	KV-Cache Management	Data	Data Preprocessing & Augment.	Networking	Gradient Compression
Architecture	Compound AI Systems	Data	Streaming Data Systems	Networking	Network Failure Modes
Architecture	RAG Systems	Data	Dataset Curation & Labeling		
Compute	GPU Compute Architecture	Deployment	Model Compression & Deploy.	Optimization	Kernel Fusion
Compute	Roofline Analysis	Deployment	Edge Inference Optimization	Optimization	Graph Compilation
Compute	Systolic Arrays & Dataflow	Deployment	Mobile Inference Deployment	Optimization	Profiling & Bottleneck Analysis
Compute	MFU & Utilization Metrics	Deployment	TinyML & MCU Deployment	Optimization	Operator Optimization
Compute	Hardware Accelerator Comp.	Deployment	Container Orchestration	Optimization	Memory Optimization Tech.
Compute	NPU & DSP (Mobile)	Deployment	A/B Testing & Rollout	Optimization	Sparsity & Pruning
		Deployment	Model Versioning & Registry	Optimization	Distillation Systems
Cross-cutting	TCO & Cost Modeling	Latency	Latency Decomposition	Parallelism	Parallelism Strategies
Cross-cutting	ML Lifecycle & Ops	Latency	Prefill/Decode Analysis	Parallelism	Data Parallelism
Cross-cutting	Responsible AI & Governance	Latency	Batching Strategies	Parallelism	Tensor Parallelism
Cross-cutting	Privacy & Federated Learning	Latency	Scheduling & Queuing	Parallelism	Pipeline Parallelism
Cross-cutting	Benchmarking & Evaluation	Latency	SLO/SLA Design	Parallelism	3D Parallelism
		Latency	Tail Latency Management	Parallelism	Expert Parallelism
Memory	VRAM Budgeting	Power	Power & Thermal Management	Precision	Quantization Systems
Memory	Memory Hierarchy Systems	Power	Energy Efficiency Metrics	Precision	Mixed Precision Training
Memory	Activation Checkpointing	Power	DVFS & Clock Management	Precision	Numerical Formats
Memory	Memory Offloading	Power	Battery & Energy (Mobile)		
Memory	Memory Fragmentation	Power	Power Budget (TinyML)	Reliability	Fault Tolerance (Training)
Memory	Unified & Virtual Memory			Reliability	Checkpointing & Recovery
Memory	Weight Memory Layout			Reliability	Silent Data Corruption
Memory	SRAM Management (TinyML)			Reliability	Monitoring & Observability
				Reliability	Graceful Degradation
					Redundancy & Replication

4.3 Knowledge Organization

The taxonomy follows the principles of faceted classification (Hjørland, 2013), where each question is independently classified on orthogonal axes rather

than slotted into a single hierarchy. This design avoids the cross-classification problems inherent in monohierarchical schemes. A question about KV-cache memory pressure during continuous batching

touches memory, latency, and deployment simultaneously, and a faceted system accommodates this naturally.

Topic selection follows user warrant (Soergel, 1985) rather than literary warrant. Topics are included because engineers need them during interviews, not because they appear frequently in the research literature. This distinction explains why the taxonomy includes `tco-cost-modeling` (rarely a research topic, always an interview topic) but excludes attention mechanism variants (heavily researched, rarely asked as systems questions).

The taxonomy is maintained against signals from industry practice and systems research (e.g., job postings, MLSys-class venues). New topics enter only if they satisfy the three inclusion filters and support questions at multiple mastery levels.

4.4 Example Topic with Edges

To illustrate the structure, consider the topic `gpu-compute-architecture`:

```
ID: gpu-compute-architecture
Area: Compute
Description: GPU execution model: warps, thread
            blocks, occupancy, Tensor Cores, and memory coalescing.
Tracks: cloud, edge
Edges:
requires → roofline-analysis
related → kernel-fusion
related → systolic-dataflow
```

The prerequisite edge encodes that understanding the roofline model is necessary before reasoning about GPU compute architecture, because determining whether a kernel is compute-bound or memory-bound requires roofline analysis. The related edges capture lateral connections: kernel fusion decisions depend on GPU architecture, and systolic arrays are an alternative accelerator execution model worth comparing.

5 Deployment Tracks

Why not simply divide the space into “training” and “inference”? Because tracks are not workload labels; they represent different **physics envelopes** that govern the napkin math. Cloud workloads are bandwidth and network-bound; Edge workloads are thermal-bound; Mobile workloads are battery-bound; and TinyML workloads are strictly SRAM-bound. A candidate who understands KV-cache sizing in the cloud cannot trivially transfer that knowledge

to an SRAM-constrained microcontroller; the physical constraints dictate entirely different engineering tradeoffs.

Table 5 summarizes the five deployment tracks. Four (cloud, edge, mobile, TinyML) correspond to distinct hardware regimes; the fifth (“global”) captures cross-cutting principles. The same concept (KV-cache management, say) requires different reasoning, different formulas, and different napkin math depending on the deployment target.

5.1 Cloud: Bandwidth and Fabric

In the cloud regime, compute is abundant but data movement is the bottleneck. An H100 GPU delivers 989 TFLOPS FP16 dense (1,979 with 2:4 structured sparsity) but only 3.35 TB/s of HBM bandwidth.⁶ The roofline ridge point at TF32 is $495/3.35 \approx 148$ FLOPs/Byte. Most LLM inference workloads, particularly the autoregressive decode phase, operate well below this ridge point, making them memory-bandwidth-bound despite the massive compute available. The dominant engineering challenges are KV-cache management, continuous batching (Yu et al., 2022) to maximize GPU utilization, and inter-node communication for distributed training where InfiniBand fabric at 400 Gb/s still becomes the bottleneck during AllReduce operations.

Cloud questions test reasoning about throughput, latency, and cost at datacenter scale. Napkin math involves TFLOPS, TB/s, and dollars per GPU-hour.

5.2 Edge: Thermal and Real-Time

Edge deployment (autonomous vehicles, robotics, industrial inspection) operates under a thermal envelope that caps sustained throughput. An NVIDIA Orin delivers 275 TOPS INT8 within a 60 W power budget, but sustained operation at maximum throughput may require thermal throttling. The dominant constraint is the intersection of compute budget and real-time deadlines. The perception pipeline must process a camera frame within 33 ms (30 fps) regardless of scene complexity, leaving no room for the tail-latency spikes that cloud systems tolerate.

Edge questions test reasoning about worst-case execution time, power budgets, and the tradeoff be-

⁶We use the H100’s TF32 Tensor Core performance of 495 TFLOPS (dense) for napkin math throughout this paper. TF32 is the default compute precision for many training and inference workloads on Hopper GPUs, and its lower throughput produces more conservative (pessimistic) estimates. The FP16 Tensor Core peak is 989 TFLOPS dense.

Table 5. Deployment tracks and physics regimes. Each track maps to distinct dominant constraints, characteristic scales, and failure modes that govern the napkin math.

Track	Dominant Constraint	Scale	Unit	Failure Mode	Napkin Math Involves
Cloud	Memory BW + network fabric	PFLOPS	TB/s	Throughput collapse	TFLOPS, TB/s, \$/GPU-hr
Edge	Thermal envelope + real-time	TOPS	Watts	Deadline miss	TOPS, Watts, milliseconds
Mobile	Battery + shared resources	TOPS/W	mAh	Battery drain	TOPS/W, mAh, background limits
TinyML	SRAM + hard real-time	MFLOPS	KB	Stack overflow	MFLOPS, KB, microseconds
Global	Cross-cutting principles	–	–	–	Depends on context

tween model accuracy and inference latency. Napkin math involves TOPS, Watts, and milliseconds.

5.3 Mobile: Battery and Resources

Mobile ML operates on shared silicon: the neural processing unit (NPU) shares power, thermal headroom, and memory bandwidth with the CPU, GPU, camera ISP, and cellular modem. A smartphone’s 4,500 mAh battery must last all day, meaning any ML workload must justify its energy cost against competing demands. The dominant constraint is TOPS per Watt. A mobile NPU may deliver 15 TOPS at 5 W, but running it continuously for on-device inference would drain the battery in under 2 hours.

Mobile questions test reasoning about energy budgets, model compression for on-device deployment, and the scheduling tradeoffs of running ML inference alongside other system tasks.

5.4 TinyML: SRAM and Real-Time

TinyML represents the most constrained physics regime. A Cortex-M4 microcontroller operates at ~100 MFLOPS with 256 KB of SRAM and no operating system (Lin et al., 2020). There is no virtual memory, no swap space, and no dynamic allocation; the model, activations, and runtime must all fit in SRAM simultaneously. The dominant constraint is hard real-time. A keyword spotting model must classify an audio frame within a fixed time budget, because missing a deadline means dropping audio samples with no recovery mechanism.

TinyML questions test reasoning about model footprint in kilobytes, fixed-point arithmetic, and the systems engineering required to run inference on bare metal. Napkin math involves MFLOPS, KB, and microseconds.

5.5 Cross-Track Variation

The track structure ensures that questions test regime-appropriate reasoning. Consider KV-cache management across tracks:

- **Cloud.** “A 70B model’s KV-cache at 128K context is ~42 GB. How many H100s do you need?” (Memory capacity reasoning at TB scale)
- **Edge.** “Your ADAS model caches 10 frames of detection history. At 30 fps on Orin’s 32 GB LPDDR5, what is the memory pressure?” (Real-time memory budget reasoning)
- **TinyML.** “Your keyword spotter needs 2 KB of rolling buffer on a 256 KB MCU. What fraction of SRAM does this consume, and what is left for the model?” (SRAM partitioning at KB scale)

All three questions test the same underlying concept (caching intermediate computation to avoid re-computation) but the physics, the formulas, and the failure modes are entirely different.

6 Mastery Levels

The 6 mastery levels map to Bloom’s revised taxonomy (Anderson et al., 2001) and are explicitly designed to correspond to industry engineering ladders. This dual mapping (cognitive depth and professional seniority) makes STAFFML practical both as an educational tool and as interview preparation calibrated to specific hiring levels. Table 6 states the Bloom’s category, typical ownership scope, and representative industry ladder band for each level.

Each level tests a qualitatively different cognitive operation. L1 requires memorization of physical constants (“HBM3 latency is ~300 ns”). L3 requires applying a formula to a novel scenario (“Calculate KV-cache size for Llama-70B at 128K context”). L6+ requires synthesizing novel architectures from first principles (“Design a scheduling policy that prevents prefill starvation under continuous batching”). At this tier the constraint relationship also becomes bidirectional: engineers not only optimize software for

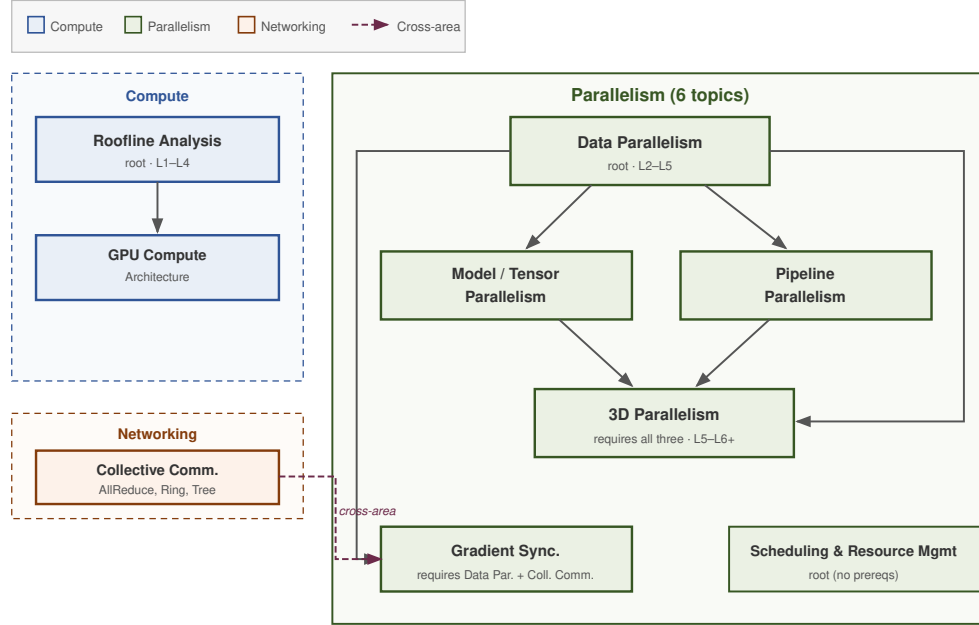


Figure 4. Prerequisite subgraph: parallelism area. Six of 87 topics showing the dependency diamond: Data Parallelism fans out to Model/Tensor Parallelism and Pipeline Parallelism, which converge on 3D Parallelism. The dashed cross-area edge from Collective Communication (networking) to Gradient Synchronization illustrates how the prerequisite graph connects competency areas. The full graph has 57 edges, maximum depth 3, and is verified acyclic.

fixed hardware, they shape hardware procurement, cluster topology, and custom silicon (“Given this workload’s arithmetic intensity, what must the ridge point of our next-generation accelerator be?”).

The industry ladder mapping is intentional, not incidental. When a hiring manager at Google prepares an L5 (Senior) interview loop, they can filter STAFFML for L4–L5 questions and know that the cognitive demands match the leveling expectations. When a candidate preparing for a Staff (L6) role wants to practice, they can target L5–L6+ questions that require the systems-level synthesis expected at that level. This alignment between Bloom’s cognitive framework and industry practice is what makes the level axis actionable for both sides of the interview table.

We adopt Bloom’s revised taxonomy ([Anderson et al., 2001](#)) over alternatives such as Webb’s Depth of Knowledge ([Webb, 1997](#)) and the SOLO taxonomy ([Biggs and Collis, 1982](#)) for three reasons. First, Bloom’s six levels align naturally with the six tiers of industry engineering ladders, providing an intuitive mapping that interviewers and candidates already understand. Second, Bloom’s cognitive verbs (remember, understand, apply, analyze, evaluate, create)

Table 6. Mastery levels. Bloom’s taxonomy mapped to industry engineering ladders. The industry column reflects typical leveling at companies like Google (L3–L7), Meta (E3–E7), and comparable organizations.

Level	Bloom’s	Scope	Industry Ladder
L1	Remember	Own a task	Intern / New Grad (L3)
L2	Understand	Own a task	Junior (L3–L4)
L3	Apply	Own a component	Mid-level (L4)
L4	Analyze	Own a component	Senior (L5)
L5	Evaluate	Own a system	Staff (L6)
L6+	Create	Own architecture	Senior Staff+ (L7+)

map directly to the cognitive operations demanded by systems interview questions, making level assignment more reliable than the content-area-dependent DOK levels. Third, the revised taxonomy’s two-dimensional structure (cognitive process $\times$ knowledge type) complements rather than conflicts with the orthogonal zone axis. Bloom’s captures depth, while zones capture cognitive mode.

The level distribution in the corpus reflects intentional design decisions. L3–L4 questions (Apply and Analyze) are the most numerous because they correspond to the cognitive sweet spot of technical interviews: complex enough to differentiate candi-

dates, but tractable within a 45-minute interview slot. L1–L2 questions serve as warm-up and screening, while L6+ questions are reserved for Staff+ interview loops where candidates are expected to synthesize novel solutions.

7 Schema and Infrastructure

The STAFFML infrastructure translates the conceptual design (four axes, 11 zones, 87 topics) into a validated data system. A single LinkML schema serves as the source of truth, generating Pydantic models for Python runtime validation, JSON Schema for CI/CD pipelines, and TypeScript types for downstream applications. This eliminates the class of bugs where one layer accepts a value that another rejects.

The prerequisite graph encoded in the schema is verified to be a directed acyclic graph (DAG), ensuring that learning paths are well-ordered. Structural consistency across the corpus is enforced by an invariant checking system that runs as a CI gate: no commit is accepted if any invariant fails. These invariants catch errors that no single-question validator can detect, such as a topic deleted from the taxonomy but still referenced by questions, a prerequisite cycle introduced by a new edge, or a topic that has accumulated an outsized share of questions. The full invariant system is described in Section 9.

8 Item Construction

Every STAFFML question is constructed for quantitative grounding and cognitive alignment with the competency model. The methodology draws on Evidence-Centered Design (ECD) (Mislevy et al., 2003) and Automatic Item Generation (AIG) (Gierl and Haladyna, 2013), prioritizing the structural properties that make questions valid assessments of ML systems expertise.

8.1 Evidence-Centered Design

Every question begins with a specification of three ECD components: (1) the *competency claim*, what the question measures, defined by its topic $\times$ zone $\times$ track $\times$ level coordinates; (2) the *evidence*, what a correct answer demonstrates, mapped to Bloom’s cognitive operation; and (3) the *task features*, the scenario elements that elicit that evidence (hardware specs, failure mode, scale). The four-axis classification is specified before question construction, ensuring coverage is intentional rather than emergent.

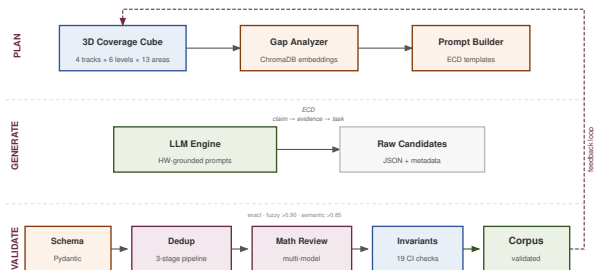


Figure 5. Item construction pipeline. Gap analysis identifies underfilled cells in the coverage space. Questions are constructed with hardware constants, Bloom’s descriptors, zone descriptions, and track constraints. Candidates pass through schema validation, multi-stage deduplication, multi-model math verification, and structural invariant checks before corpus integration.

8.2 Hardware Grounding

Figure 5 summarizes the end-to-end construction path: gap analysis targets underfilled coverage cells, each item is assembled from taxonomy and hardware-grounded prompts, and only candidates that pass validation, deduplication, and math verification enter the corpus.

Each question is grounded in five components drawn from the schema:

- **Topic description** from the taxonomy: what concept to test
- **Zone description**: how to test it (which skills to exercise)
- **Track constraints**: which physics regime applies
- **Level expectations**: which Bloom’s cognitive operation to target
- **Hardware constants**: real specifications drawn from vendor datasheets (NVIDIA Corporation, 2022a,b; Advanced Micro Devices, 2023; Arm Limited, 2020) (H100: 80 GB HBM3, 3.35 TB/s, 495 TFLOPS TF32 dense; MI300X: 192 GB HBM3, 5.3 TB/s; Orin: 275 TOPS INT8; Cortex-M4: 168 MHz, 256 KB SRAM)

All questions are grounded in the content and formulas of the *Machine Learning Systems* textbook (Reddi, 2026b,a), which provides the hardware specifications, architectural principles, and quantitative frameworks that serve as the foundation for question construction. The hardware constants are drawn from the MLSysIM infrastructure modeling framework (Reddi, 2026c), ensuring that all questions reference the same specifications and that every napkin math answer can be independently verified against the framework. This prevents specification drift, a

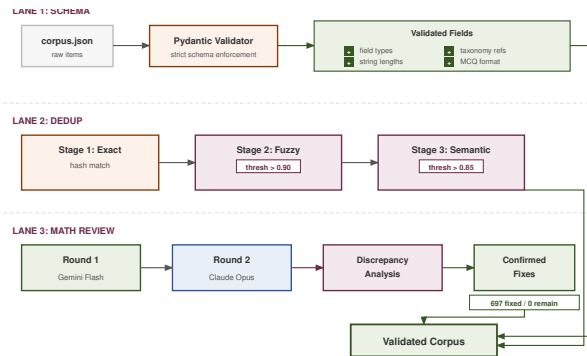


Figure 6. Multi-stage quality assurance pipeline. Candidate questions pass through schema validation (Pydantic), three-stage deduplication (exact, fuzzy, semantic), and multi-model math verification (Gemini Flash broad review + Claude Opus deep review). Confirmed errors are corrected; duplicates are archived, never deleted.

systematic error pattern where different questions cite different numbers for the same hardware, undermining the internal consistency that makes napkin math trustworthy. Hardware constants require maintenance as new accelerator generations ship; the methodology is designed to be timeless even as specific numbers evolve.

8.3 Gap-Driven Construction

Gap analysis identifies underfilled cells in the topic $\times$ zone $\times$ track space; construction prioritizes those cells. The coverage cube contains 233 semantically valid topic-track cells; summary statistics appear in Section 10.

8.4 LLM-Assisted Generation

This raises an apparent paradox: if STAFFML tests the systems reasoning that LLMs struggle with, how can LLMs generate the questions? LLMs struggle to reason about novel systems from first principles, but they generate structured variations over fixed physical constraints effectively when taxonomy coordinates, hardware constants, and task templates are supplied by the schema and MLSysIM. Generation is LLM-assisted; correctness is not taken on trust from any single model.

The value lies not in the drafting step but in the quality infrastructure that enforces physics-grounding and validates every item before publication. Prompts fix the topic, zone, track, and level, inject the five grounding components above, and anchor scenarios in textbook material (Reddi, 2026b,a) so items stay tied to shared

specifications rather than free-form model recall. The same coordinates and constants yield reproducible structure across drafts; verification (Section 9) is what makes the bank safe to use.

The methodology treats LLMs as structured content generators, not domain experts. Quantitative grounding comes from injected constants and schema constraints, not from training data. Three properties make the generated questions robust. First, every question requires reasoning from specific hardware constants that the candidate must internalize, not retrieve from a prompt. Second, the multi-model verification pipeline (Section 9) catches factual errors and arithmetic mistakes. Third, the depth chain structure (Section 3.5) ensures questions form coherent cognitive progressions that resist template-based generation.

Every item undergoes the same quality path: schema validation, deduplication, and independent math verification (Section 9). Together, Figure 5 and Figure 6 illustrate the two halves of this continuous end-to-end generation and validation pipeline. A community contribution path for industry-sourced questions is planned to widen coverage of production failure modes.

9 Quality Assurance

Quality assurance is the second half of the item construction pipeline (Figure 6). Every draft question must pass three stages (schema validation, multi-stage deduplication, and multi-model math verification) before entering the corpus. Cross-file structural consistency is enforced by a 19-check invariant system.

9.1 Schema Validation

Every question is validated against a strict Pydantic schema (generated from the LinkML definition) with both field-level and cross-question rules:

- **Field constraints.** Minimum character lengths (scenario ≥ 50 , napkin_math ≥ 10 and must contain a digit), MCQ must have exactly 4 options with correct_index in $[0, 3]$
- **Taxonomy constraints.** topic $\in \{87 \text{ curated IDs}\}$, zone $\in \{11 \text{ ikigai zones}\}$, track $\in \{5 \text{ tracks}\}$, level $\in \{L1 \dots L6+\}$
- **Uniqueness constraints.** No duplicate IDs, no duplicate (track, title) pairs among published questions

Table 7. Corpus summary. Key statistics of the STAFFML vault.

Metric	Value	Track	Questions
Published questions	9,199	Cloud	4,122 (44.8%)
Curated topics	87	Edge	1,968 (21.4%)
Competency areas	13	Mobile	1,641 (17.8%)
Ikigai zones	11	TinyML	1,163 (12.6%)
Mastery levels	6	Global	305 (3.3%)
Prerequisite edges	123		
Depth chains	979	Total	9,199
Schema	LinkML	Validated	100.0%

- **Linguistic checks.** No trailing quotes, no mark-down formatting artifacts, no placeholder text

Field coverage across the corpus is 100% for all required fields: topic, zone, competency_area, napkin_math, common_mistake, realistic_solution, and bloom_level. Under the multi-model math-verification pipeline (Section 9.3), 100.0% of published questions carry the validated flag.

9.2 Multi-Stage Deduplication

We employ a three-stage deduplication pipeline inspired by web-scale techniques (Broder, 1997):

1. **Exact match.** Lowercased, whitespace-stripped scenario comparison. Catches copy-paste duplicates.
2. **Fuzzy match.** `difflib.SequenceMatcher > 0.90` within same (track, topic) groups. Catches minor rewordings.
3. **Semantic match.** Sentence-BERT (Reimers and Gurevych, 2019) cosine similarity > 0.85 via ChromaDB. Catches same-concept-different-words duplicates.

Confirmed duplicates are archived (status set to “archived”), never deleted, preserving the audit trail, a practice adopted from large-scale community assessment management (AnKing Team, 2024).

9.3 Math Verification

Napkin math is the soul of STAFFML and its most error-prone component. Hardware specifications change across generations, unit conversions cascade, and order-of-magnitude errors slip in easily. Verification combines automated checks with independent recomputation against MLSysIM constants: a broad pass flags arithmetic, specification, and unit issues; a second pass stress-tests logic; a final stage recomputes calculations from first principles so answers do

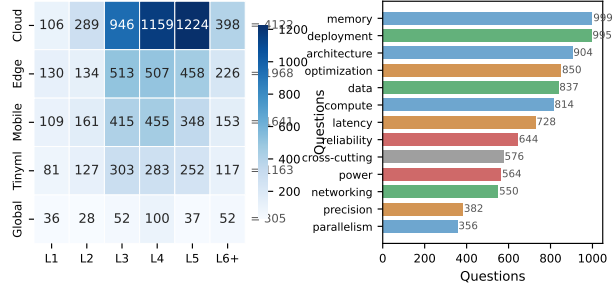


Figure 7. Track $\times$ Level heatmap. Cloud L3–L5 cells are densest, reflecting industry emphasis on mid-to-senior systems reasoning.

not depend on a single model’s draft. Discrepancies are reconciled before publication. Recent work on LLM-as-judge evaluation (Zheng et al., 2023) supports using cross-model agreement to improve reliability over single-model review. The corpus also allows community error reporting after release.

The full published set was checked under this pipeline; items with incorrect or inconsistent mathematics relative to the stated hardware scenario were revised or dropped before release. Readers should still treat any large bank as living software: constants evolve with new accelerators, and formal psychometric validation remains future work (Section 14).

9.4 LLM Failure Modes: A Taxonomy

Draft models exhibit characteristic failure modes that casual reading misses: confusing accelerator specifications between product lines (e.g., mis-stating memory bandwidth), dropping factors in collective-communication or roofline calculations, swapping unit prefixes so digits look right but magnitude is wrong, applying training-only concepts to inference scenarios, and writing scenarios whose stated parameters contradict the implied network or memory model. Cross-model recomputation against a single constants source catches these errors; it is not redundant with generation.

9.5 Structural and Semantic Invariants

Beyond per-question validation, STAFFML enforces 19 strict invariants across the dataset. While some are standard referential integrity checks (e.g., ensuring the prerequisite graph is acyclic and all foreign keys resolve), the system’s true value lies in enforcing domain-specific semantic and physics constraints:

- **Physics applicability.** A question’s topic-track pair must strictly adhere to the physics exclusion matrix (e.g., an LLM generating an RDMA ques-

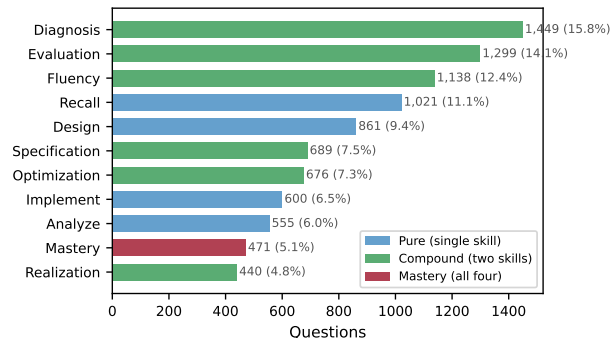


Figure 8. Zone distribution. Compound zones (diagnosis, evaluation, fluency) dominate by design: they most sharply differentiate Staff-level candidates.

tion for a TinyML track is immediately rejected at commit-time).

- **Cognitive affinity.** A question’s zone must be cognitively compatible with its Bloom’s level. For example, a question assigned to the “Evaluation” zone is structurally rejected if its difficulty is tagged as L1 (Remember) or L2 (Understand).
- **Chain progression.** Questions linked in a depth chain must strictly monotonically increase in Bloom’s level; a chain cannot regress from L4 back to L2.
- **Coverage health.** The distribution cannot collapse: no single topic may exceed 10% of the corpus, and all defined zones must maintain a minimum population threshold to prevent model drift toward easy-to-generate topics.

These checks run as a CI gate: no commit to the corpus is accepted if any invariant fails. A traditional LLM validation pass might miss that an L1 Evaluation question is an oxymoron, but the invariant system guarantees the structural and cognitive coherence of the entire 9,199-question bank.

10 Coverage Analysis

Coverage analysis examines how questions distribute across the four classification axes, highlighting where the corpus is strong and where more items would improve balance.

10.1 Track Distribution

The track distribution reflects both industry demand and regime breadth. Cloud is the densest track, consistent with datacenter LLM serving and distributed training. Edge, mobile, and TinyML carry substantial mass so on-device constraints appear throughout the bank. The global track holds cross-platform concep-

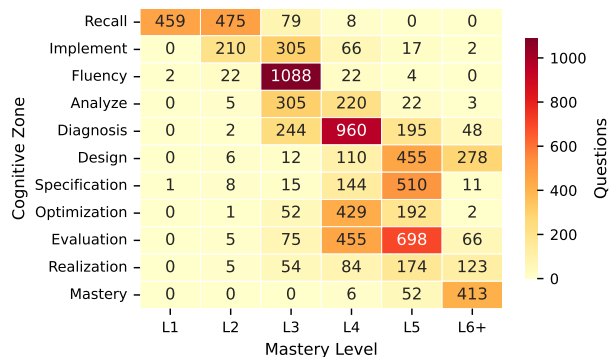


Figure 9. Zone × Level heatmap. Each ikigai zone concentrates at its affinity Bloom levels.

tual items. Table 7 reports the exact distribution. Figure 7 shows the same corpus as a track × mastery-level heatmap: Cloud questions at L3–L5 form the densest cells, consistent with industry demand for mid-to-senior backend systems engineers.

10.2 Zone Distribution

Figure 8 gives per-zone counts across the eleven ikigai zones. Compound zones (diagnosis, evaluation, fluency) dominate the corpus, which is intentional: they most sharply differentiate Staff-level candidates. All 11 zones now exceed 250 questions, with diagnosis (1,449), evaluation (1,299), and fluency (1,138) leading the distribution.

10.3 Level and Bloom’s Distribution

The Bloom’s distribution across the corpus reflects the cognitive demands of Staff-level interviews: Analyze (3,105 questions, 33.8%), Evaluate (2,001, 21.8%), Apply (1,978, 21.5%), Create (875, 9.5%), Understand (677, 7.4%), and Remember (563, 6.1%). The emphasis on Analyze and Evaluate reflects the higher-order reasoning required at the Staff level. Figure 10 complements these category totals by stacking item formats (conceptual, calculation, design, diagnosis, tradeoff, optimization) across Bloom levels.

10.4 Multi-Axis Coverage

Figures 9 and 10 summarize how zones, Bloom levels, and item formats line up under the authoring priors. The zone × level heatmap shows each ikigai zone clustering at its intended difficulty: recall peaks at L1–L2, fluency anchors at L3, diagnosis centers on L3–L4, and mastery is confined to L6+. The chart shows the same story at the item level: conceptual prompts dominate junior levels, calculation peaks at L3, and open-ended design dominates L5–L6+.

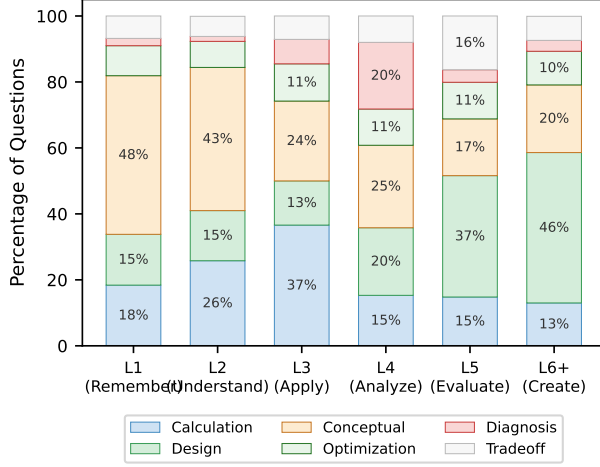


Figure 10. Question format distribution by Bloom's level. Conceptual questions dominate L1–L2, calculation peaks at L3, design dominates L5–L6+.

The three-dimensional coverage space of topic $\times$ zone $\times$ track contains $87 \times 11 \times 4 = 3,828$ cells (excluding the small “global” track). Not all cells are semantically meaningful: a TinyML question about InfiniBand fabric topology, or a cloud question about MCU duty cycling, has no physical substrate to reason about. We define an *applicability matrix* that excludes 83 topic–track pairs where the underlying concept has no physical realization on that hardware tier. Each exclusion carries a physics-grounded justification (e.g., “RDMA requires InfiniBand/RoCE NICs; edge uses Ethernet/WiFi”). The remaining 233 applicable pairs yield 2,563 cells (233×11 zones), reducing coverage by 27%.

Table 8 tabulates every topic–track pair; Table 9 aggregates those exclusions into summary counts. Cloud-only topics (11) involve datacenter-scale infrastructure such as pipeline parallelism, RDMA transport, and container orchestration, all of which require multi-device interconnects absent from edge, mobile, and TinyML platforms. Conversely, MCU compute constraints, OTA firmware updates, and duty cycling exclude cloud because their engineering challenges (KB-scale memory, battery harvesting, real-time deadlines) have no analogue in datacenter environments. The largest exclusion category is TinyML (28 topics excluded), reflecting the fundamental constraint that microcontrollers with 256 KB SRAM cannot instantiate concepts like KV-cache management, mixture-of-experts routing, or activation checkpointing.

Reading the grid, checkmarks indicate a meaningful physical substrate for interview questions and crosses mark physics-grounded exclusions. The vi-

sual pattern aligns with Table 9: parallelism and networking skew cloud-only on constrained tiers, while precision and cross-cutting topics are largely universal. The TinyML column carries the most exclusions, reflecting that microcontrollers with 256 KB SRAM cannot instantiate concepts requiring GB-scale memory.

11 Example Questions

This section presents six questions that illustrate how the four-axis classification captures distinct cognitive acts. To isolate what each axis contributes, we vary one axis at a time. The first five examples hold track constant (cloud) and progress through five zones (fluency, diagnosis, specification, evaluation, mastery) at increasing levels (L3–L6+), so the reader can observe how the cognitive demand changes independently of the physics regime. The sixth example then holds topic constant (quantization) and varies track across cloud, edge, and TinyML, revealing how the same concept transforms under different hardware constraints.

11.1 Fluency: KV-Cache Sizing

Fluency tests whether a candidate can perform napkin math from memory, recalling both the formula and the hardware specifications, then executing the arithmetic. It is the signature skill tested in Staff-level phone screens.

vram-budgeting
fluency • cloud • L3

Scenario: You need to increase your LLM’s context window from 4K to 128K tokens. The model weights fit perfectly in your 80 GB VRAM. What hidden memory cost will cause your node to OOM during generation?

Common mistake: “Activations will use more memory.” Activations matter during training, but during inference the dominant hidden cost is something else entirely.

Napkin math: For Llama-70B (80 layers, 8 KV-heads via grouped-query attention (Ainslie et al., 2023), head_dim=128) at 128K tokens in FP16:

$$\underbrace{2}_{K,V} \times \underbrace{80}_{\text{layers}} \times \underbrace{8}_{\text{heads}} \times \underbrace{128}_{\text{dim}} \times \underbrace{128K}_{\text{tokens}} \times \underbrace{2}_{\text{bytes}} \approx \mathbf{41.9\text{ GB}}$$

Combined with 140 GB of weights, one 128K-context request requires ~182 GB. This spills over an 80 GB NVIDIA H100 (requiring 3 GPUs) but fits entirely on a single 192 GB AMD MI300X.

Answer: The KV-cache. It grows linearly with sequence length and, at 128K context, can exceed the model weights themselves.

Table 8. Applicability matrix. Each row is a topic; the four marks are Cloud, Edge, Mobile, and TinyML. **Bold** rows are category headers. Read across a row for a topic’s tracks; read each of the three topic columns top to bottom (rows do not align across columns). ✓: applicable; ✗: excluded. Of 87×4 pairs, 233 are applicable and 83 excluded.

Topic	Cloud	Edge	Mobile	TinyML	Topic	Cloud	Edge	Mobile	TinyML	Topic	Cloud	Edge	Mobile	TinyML
Architecture (7)					Deployment (7)					Optimization (7)				
Attention Scaling & Variants	✓	✓	✓	✓	A/B & Rollout Strategies	✓	✓	✓	✗	Graph Compilation & Optimization	✓	✓	✓	✓
CNN Efficient Design	✓	✓	✓	✓	Compound AI Systems	✓	✓	✓	✗	IO-Aware Attention	✓	✗	✗	✗
Encoder-Decoder Tradeoffs	✓	✓	✓	✗	Container & Cluster Orchestration	✓	✗	✗	✗	Kernel & Operator Fusion	✓	✓	✓	✗
Mixture of Experts	✓	✗	✗	✗	MLOps Lifecycle	✓	✓	✓	✗	Knowledge Distillation	✓	✓	✓	✓
Model Size Estimation	✓	✓	✓	✓	Model Format & Runtime Conversion	✗	✓	✓	✓	Operator Scheduling	✓	✓	✓	✓
Neural Architecture Search	✓	✓	✓	✓	Model Serving Infrastructure	✓	✓	✓	✗	Pruning & Sparsity	✓	✓	✓	✓
Transformer Systems Cost	✓	✓	✓	✓	OTA & Firmware Updates	✗	✓	✗	✓	Speculative Decoding	✓	✓	✓	✗
Compute (6)					Latency (6)					Parallelism (6)				
Accelerator Comparison	✓	✓	✓	✓	Batching Strategies	✓	✓	✗	✗	3D Parallelism	✓	✓	✗	✗
Compute Cost Estimation	✓	✓	✓	✓	Latency Decomposition	✓	✓	✓	✓	Data Parallelism	✓	✓	✗	✗
GPU Compute Architecture	✓	✓	✗	✗	Profiling & Bottleneck Analysis	✓	✓	✓	✓	Gradient Synchronization	✓	✗	✗	✗
MCU Compute Constraints	✗	✗	✗	✓	Queueing Theory	✓	✓	✗	✗	Model & Tensor Parallelism	✓	✗	✗	✗
Roofline Analysis	✓	✓	✓	✓	Real-Time Deadlines	✗	✓	✓	✓	Pipeline Parallelism	✓	✗	✗	✗
Systolic Arrays & Dataflow	✓	✓	✓	✓	Tail Latency & SLAs	✓	✓	✗	✗	Scheduling & Resource Management	✓	✗	✗	✗
Cross-Cutting (5)					Memory (8)					Power (5)				
Differential Privacy	✓	✓	✓	✗	Activation Memory & Checkpointing	✓	✓	✗	✗	Datacenter Efficiency	✓	✗	✗	✗
Fairness & Bias Evaluation	✓	✓	✓	✗	DMA & Data Movement	✓	✓	✓	✓	Duty Cycling & Energy Harvesting	✗	✓	✓	✓
Federated Learning	✓	✓	✓	✓	KV-Cache Management	✓	✗	✓	✗	Energy Per Operation	✓	✓	✓	✓
Responsible AI & Governance	✓	✓	✓	✓	Memory Hierarchy Design	✓	✓	✓	✓	Power Budgeting	✓	✓	✓	✓
TCO & Cost Modeling	✓	✓	✓	✓	Memory Pressure Management	✓	✓	✓	✗	Thermal Management	✓	✓	✓	✗
Data (7)					Memory-Mapped Inference	✓	✓	✓	✗	Precision (3)				
Data Efficiency & Selection	✓	✓	✓	✗	Tensor Arena Planning	✓	✓	✗	✓	Extreme Quantization	✓	✓	✓	✓
Data Pipeline Engineering	✓	✓	✓	✓	VRAM Budgeting	✓	✓	✓	✓	Mixed-Precision Training & Inference	✓	✓	✓	✓
Data Quality & Validation	✓	✓	✓	✗	Networking (6)					Quantization Fundamentals	✓	✓	✓	✓
Dataset Curation & Labeling	✓	✓	✓	✓	Collective Communication	✓	✓	✓	✓	Reliability (6)				
Feature Store & Feature Engineering	✓	✗	✗	✗	Congestion & Flow Control	✓	✗	✗	✗	Adversarial Robustness & Security	✓	✓	✓	✓
Storage Format & Selection	✓	✓	✗	✗	Interconnect Topology	✓	✗	✓	✗	Distribution & Drift Detection	✓	✓	✓	✗
Streaming & Real-Time Ingestion	✓	✓	✓	✓	Load Balancing & Routing	✓	✓	✗	✗	Fault Tolerance & Checkpointing	✓	✓	✓	✗
					Network Bandwidth Bottlenecks	✓	✗	✓	✗	Graceful Degradation	✓	✓	✓	✓
					RDMA & High-Performance Transport	✓	✗	✗	✗	Monitoring & Observability	✓	✓	✓	✓
										Safety & Certification	✗	✓	✓	✓

Table 9. Applicability matrix summary. Topic-track exclusions grouped by physics rationale. Each exclusion means the concept has no physical substrate on that hardware tier.

Pattern	Count	Example topics
Cloud-only (11 topics)	33 excl.	TP/PP, RDMA, MoE, K8s
TinyML-excluded	28 excl.	KV-cache, kernel fusion, MoE
No-cloud (device-only)	6 topics	duty cycling, OTA, safety cert
TinyML-only	1 topic	MCU compute constraints
Selective (2–3 tracks)	18 excl.	flash-attn, systolic, tail-lat
Total excluded	83 pairs	83×11 cells removed
Applicable	233 pairs	2,563 cells targeted

This is a fluency question. The candidate must recall the KV-cache formula and execute the arithmetic from memory, producing a specific number. No architectural design is required; no tradeoff analysis is asked for. The cognitive act is rapid quantitative calculation, the signature skill of the fluency zone.

Timeless principle: KV-cache memory scales linearly with sequence length and quadratically with attention head count. The specific hardware capacities change (e.g., 80 GB vs. 192 GB); the scaling law does not.

11.2 Diagnosis: MFU Collapse

Diagnosis tests whether a candidate can identify root causes from observable symptoms. The candidate must recall hardware specifications and reason about which bottleneck explains the observed behavior. This is the “3 AM on-call” skill.

roofline-analysis
diagnosis • cloud • L4

Scenario: Your production LLM serving cluster reports 40% MFU during the decode phase. The H100s are rated at 495 TFLOPS TF32 dense. Your team assumes the GPUs are underclocked. Diagnose the actual root cause.

Common mistake: Blaming hardware or driver issues. The decode phase of autoregressive generation is inherently memory-bandwidth-bound, not compute-bound.

Napkin math: During decode, each token requires reading the full model weights ($2 \times 70B = 140$ GB in FP16) but performs only $2 \times 70B = 140$ GFLOPs of compute. Arithmetic intensity = $140 \text{ GFLOPs} / 140 \text{ GB} = 1 \text{ FLOP/Byte}$, far below the H100’s ridge point of $\sim 148 \text{ FLOPs/Byte}$ (dense). The workload is bound by the 3.35 TB/s memory bandwidth ceiling, not the 495 TFLOPS compute ceiling.

Answer: The decode phase is memory-bandwidth-bound. MFU measures compute utilization, but the bottleneck is data movement. The 40% figure is not a deficiency; the

GPUs are performing as expected given the workload’s arithmetic intensity.

This is a diagnosis question. The candidate must recall hardware specifications (H100 bandwidth, ridge point) and analyze why those specs explain the observed symptom (low MFU). The cognitive act is “identify and explain a failure from first principles.”

Timeless principle: When arithmetic intensity falls below the hardware ridge point, adding compute does not help. Workloads shift from compute-bound to memory-bound as batch size decreases.

11.3 Specification: Latency-Based Serving

Specification tests whether a candidate can translate quantitative constraints into a concrete architecture. The candidate must recall hardware capabilities and design a system that satisfies a hard performance requirement. This is the core skill of system design interviews.

model-serving-infrastructure
specification • cloud • L5

Scenario: Your team must serve a 70B-parameter LLM with P99 time-to-first-token (TTFT) under 200 ms. The model is deployed on H100 GPUs with NVLink. Specify the minimum serving configuration.

Common mistake: Focusing only on model parallelism degree while ignoring the prefill compute budget.

Napkin math: Prefill FLOPs for a 2,000-token prompt:

$$\underbrace{2}_{\text{MACs}} \times \underbrace{70B}_{\text{params}} \times \underbrace{2,000}_{\text{tokens}} = 280 \text{ TFLOPs}$$

A single H100 at 495 TFLOPS TF32 dense completes this in $280/495 \approx 566$ ms, far exceeding the 200 ms SLO. With 4-way tensor parallelism, prefill drops to ~ 142 ms, fitting the budget with margin for scheduling overhead. Model weights (140 GB FP16) require 2 GPUs minimum for capacity. The configuration is 4× H100 with tensor parallelism, continuous batching, and chunked prefill.

Answer: 4-way tensor parallelism across H100s with NVLink. The constraint is prefill compute, not memory capacity: even though the model fits on 2 GPUs, the latency SLO demands additional parallelism.

This is a specification question. The candidate must recall hardware capabilities (H100 TFLOPS, NVLink bandwidth) and design a serving architecture that satisfies a quantitative constraint (P99 TTFT < 200 ms). The cognitive act is “translate constraints into architecture.”

11.4 Evaluation: The Communication Tax

Evaluation tests whether a candidate can weigh competing architectures and justify a choice with quantitative reasoning. The candidate must recall hardware capabilities and analyze which design keeps the most expensive communication off the critical path. This is the core skill of architecture review.

parallelism-strategies
evaluation • cloud • L5

Scenario: You are training a 175B-parameter model across 1,024 GPUs. Nodes are connected by 400 Gb/s InfiniBand; GPUs within a node share 3.2 TB/s NVLink. Evaluate whether pipeline parallelism (PP) or tensor parallelism (TP) should cross the inter-node fabric to minimize the communication tax.

Common mistake: Treating all bandwidth as equivalent and applying TP across nodes to avoid pipeline bubbles, ignoring the latency penalty of InfiniBand AllReduce on the critical path.

Napkin math: TP requires a blocking AllReduce for every layer ($2 \times$ activation size). Over 400 Gb/s InfiniBand, these frequent synchronizations dominate step time and collapse scaling efficiency; over 3.2 TB/s NVLink they are easily absorbed. PP requires only point-to-point transfer of activations at stage boundaries, a fraction of the bandwidth that 400 Gb/s InfiniBand can comfortably handle.

Answer: TP stays within the node (over NVLink); PP crosses nodes (over InfiniBand). The communication tax dictates the parallelism topology: high-frequency, latency-sensitive synchronizations belong on the high-bandwidth domain, while lower-frequency, bandwidth-tolerant transfers can span the slower fabric.

This is an evaluation question. The candidate must recall the bandwidth and latency gap between NVLink and InfiniBand and analyze which parallelism strategy keeps the blocking synchronizations on the fast fabric. The cognitive act is “weigh competing designs against fixed constraints and justify the choice.”

Timeless principle: The fastest interconnect in a hierarchy absorbs the most frequent communication. As long as intra-node bandwidth exceeds inter-node bandwidth, tensor-parallel AllReduce belongs inside the node regardless of which accelerator generation is in use.

11.5 Mastery: Continuous Batching Stall

Mastery requires all four skills simultaneously: recalling specs, analyzing the failure mode, designing an architectural fix, and sizing the solution with napkin math. These questions simulate the complete Staff-level interview loop.

batching-strategies
mastery • cloud • L6+

Scenario: You implement continuous batching for an LLM endpoint. Under low load, TTFT is 100 ms. Under high load, throughput triples, but TTFT spikes to over 4 seconds even though token generation speed remains fast. Diagnose the root cause, design a fix, and estimate the resource cost of your solution.

Common mistake: Assuming that because continuous batching optimizes throughput, it automatically guarantees low latency for incoming requests.

Napkin math: A 2,000-token prefill on a 70B model: $2 \times 70B \times 2,000 = 280$ TFLOPs. An H100 at 495 TFLOPS dense needs ~ 566 ms. If 4 users send prompts simultaneously, the 4th waits over 1.5 seconds before prefill even begins.

Answer: Prefill starvation. The scheduler must chunk or defer prefills to prevent blocking the decode phase, but aggressive deferral under high load creates a queue that destroys TTFT. The fix is chunked prefill with priority scheduling (Agrawal et al., 2024), but this requires overprovisioning compute by $\sim 30\%$ to absorb the prefill bursts without queuing.

This is a mastery question. The candidate must recall hardware specs (H100 TFLOPS), analyze the scheduling conflict (prefill blocks decode), design a solution (chunked prefill with priority), and execute the napkin math to size the required overprovisioning. All four skills are exercised in a single question.

Timeless principle: In multiplexed serving, prefill and decode compete for the same compute; any scheduling policy must bound prefill latency to prevent decode starvation.

11.6 Cross-Track Comparison: Quantization

The preceding four examples held track constant to isolate zone differences. We now hold topic constant (quantization) and vary the track, showing how the same concept demands different reasoning under different physics regimes:

- **Cloud** (L4, evaluation). “GPTQ (Frantar et al., 2023) vs. AWQ (Lin et al., 2024) for serving a 70B model on an Intel Gaudi 3 cluster. Which quantization scheme yields higher throughput at FP16-equivalent perplexity? Justify with arithmetic intensity analysis.” The candidate evaluates two compression strategies against datacenter-scale metrics (tokens/second/dollar).
- **Edge** (L3, fluency). “Quantize a YOLOv8 detection model from FP32 to INT8 for Orin deployment. Calculate the memory savings and estimate the latency improvement given Orin’s

INT8 TOPS vs. FP32 TFLOPS.” The candidate performs napkin math within a thermal and real-time envelope.

- **TinyML** (L5, specification). “Your 256 KB MCU must run a keyword spotting model. The FP32 model is 800 KB. Specify a quantization strategy (INT8, INT4, or binary) that fits in SRAM while maintaining >90% accuracy. Justify each precision choice for weights vs. activations.” The candidate designs a mixed-precision scheme under extreme memory constraints.

All three questions test quantization knowledge, but the physics regime determines the constraints, the formulas, and the engineering tradeoffs. A candidate who can answer the cloud question may struggle with the TinyML variant, because the reasoning transfers only partially across regimes.

Together, these six questions demonstrate the classification’s two independent degrees of freedom. Holding track constant reveals qualitatively different cognitive acts: fluency tests whether you can compute a number, diagnosis whether you can explain an observation, specification whether you can translate constraints into architecture, and mastery whether you can synthesize a complete solution. Holding topic constant then reveals that the same concept demands different formulas, different constraints, and different engineering tradeoffs depending on the physics regime.

Future work includes questions that provide partial information, requiring candidates to identify what additional data they need, a skill that tests meta-cognitive reasoning about systems diagnostics. Such incomplete-information questions would assess whether candidates know what they do not know, a hallmark of Staff-level engineering judgment.

12 Practical Applications

The zone model translates directly into interview loop design. A typical four-round interview should cover at least four zones to assess breadth of systems reasoning:

- **Phone screen.** Fluency questions. Can the candidate rapidly estimate performance bounds from memory? This is the fastest signal for mechanical sympathy.
- **System design.** Specification or Design questions. Can the candidate architect a system under quantitative constraints?

- **Debugging.** Diagnosis questions. Can the candidate root-cause a failure from observable symptoms and hardware specifications?
- **Architecture review.** Evaluation or Mastery questions. Can the candidate analyze tradeoffs and justify design decisions with quantitative reasoning?

A “strong hire” signal at Staff level is consistent strength in compound zones, especially diagnosis and evaluation. A “no hire” signal is strong recall but weakness across all compound zones: the profile of a memorizer rather than a reasoner. No interview loop should test only pure zones, as these fail to exercise the integrative reasoning that distinguishes Staff-level engineers from mid-level ones.

Not all 11 zones are equally important for interview design. The five compound zones (diagnosis, specification, fluency, evaluation, and optimization) and mastery carry the most signal for Staff+ assessment. The four pure zones (recall, analyze, design, quantify) serve primarily as corpus organization categories and warm-up question pools; they are the building blocks from which compound zones are constructed, not assessment targets in their own right. Realization (design $\cap$ quantify), while theoretically defined, appears infrequently in practice because most sizing questions also require recall, making them fluency or specification questions instead.

13 Related Work

STAFFML draws on three bodies of work: interview preparation platforms, ML systems research that provides the technical content, and educational measurement theory that grounds the assessment methodology.

13.1 Interview Platforms

LeetCode ([LeetCode Inc., 2024](#)) hosts 2,800+ algorithmic coding problems with community-driven difficulty calibration and is the dominant platform for software engineering interview preparation. NeetCode ([NeetCode, 2024](#)) curates 150 problems organized by algorithmic pattern with explicit difficulty progression. Grind 75 ([Tay, 2024](#)) ranks 169 problems by interview frequency and pattern coverage. All three platforms excel at algorithmic coding assessment but provide no coverage of systems reasoning, hardware constraints, or napkin math, the cognitive skills that ML systems interviews demand.

Table 10. Interview preparation landscape. Existing platforms optimize for algorithmic coding or conceptual systems knowledge; STAFFML is the first to combine quantitative rigor with ML systems focus and a structured taxonomy.

Platform	Items	Systems	Quantitative	HW-Grounded	Cognitive Model
LeetCode	2,800+	–	✓	–	Easy / Med / Hard
NeetCode	150	–	✓	–	Pattern-based
Grind 75	169	–	✓	–	Frequency rank
Huyen (DMLS)	Conceptual	✓	–	–	–
STAFFML	9,199	✓	✓	✓	11 zones × 6 levels

Huyen’s *Designing Machine Learning Systems* (Huyen, 2022) covers ML system design broadly (data engineering, model serving, monitoring) and is widely used for interview preparation. It does not include quantitative practice problems or a structured difficulty progression. Candidates who study the book understand what a feature store is but cannot calculate the memory bandwidth required to serve a 70B model at target throughput.

Table 10 summarizes the positioning. STAFFML addresses the gap between these extremes: the quantitative rigor of LeetCode-style practice (every question has a definite numerical answer) combined with the systems focus of conceptual interview guides (every question is grounded in real hardware constraints). The four-axis classification adds structured difficulty progression and cognitive-mode coverage that no existing platform provides.

13.2 LLM Engineering Benchmarks

Recent benchmarks like HumanEval (Chen et al., 2021) and SWE-bench (Jimenez et al., 2024) evaluate the coding and bug-fixing capabilities of LLM agents. These benchmarks focus on algorithmic generation and repository-level software engineering; they do not assess whether an agent understands the physical deployment constraints of the code it writes. STAFFML fills this gap, serving not only as an interview bank for human engineers but as a rigorous benchmark for evaluating the mechanical sympathy of LLM agents.

STAFFML also offers a structural defense against data contamination, a persistent problem in LLM evaluation where models memorize static benchmarks from their training data. Because STAFFML questions are generated programmatically using hardware constants (Section 8), an evaluation suite can randomize these constants (e.g., changing an A100’s bandwidth from 2.0 TB/s to an arbitrary 2.4 TB/s) to ensure the LLM agent is actually per-

forming the physical reasoning and napkin math, rather than retrieving a memorized answer.

13.3 ML Systems Research

STAFFML questions are grounded in seminal ML systems research. Megatron-LM (Shoeybi et al., 2019) introduced tensor parallelism for training large language models, providing the foundation for questions about parallelism strategies and communication overhead. GPipe (Huang et al., 2019) established pipeline parallelism and the bubble fraction analysis that appears in realization-zone questions. ZeRO (Rajbhandari et al., 2020) and DeepSpeed (Rasley et al., 2020) eliminated memory redundancy in data parallelism, motivating questions about optimizer state partitioning and memory budgeting.

FlashAttention (Dao et al., 2022) demonstrated IO-aware algorithm design, reducing memory reads/writes from quadratic to linear in sequence length, and provides the canonical example of how understanding the memory hierarchy yields algorithmic improvements invisible to complexity analysis alone. vLLM (Kwon et al., 2023) introduced PagedAttention for KV-cache management, directly inspiring our fluency and diagnosis questions about memory fragmentation and serving throughput. Orca (Yu et al., 2022) pioneered continuous batching (iteration-level scheduling), which motivates our mastery-zone questions about prefill starvation and scheduling tradeoffs.

MLPerf (Mattson et al., 2020; Reddi et al., 2020) established standardized benchmarks for measuring ML system performance: training time, inference latency, and throughput across hardware platforms. The distinction matters: MLPerf measures how well systems perform; STAFFML measures how well humans reason about systems performance. MLPerf’s measurement methodology (standardized workloads, reproducible configurations, hardware-specific baselines) informs the quantitative grounding of STAFFML questions, and several questions

Table 11. Source material mapping. Each seminal system provides the quantitative content for questions in specific zones. The mapping illustrates how STAFFML operationalizes research into assessment.

System	Concept Tested	Primary Zones	Example Task
Megatron-LM GPipe	Tensor parallelism Pipeline bubbles	Realization, Fluency Realization, Optimization	Size the TP degree for 70B on $8 \times H100$ Calculate bubble fraction for k micro-batches
ZeRO / DeepSpeed	Memory partitioning	Fluency, Diagnosis	Budget optimizer state across N GPUs
FlashAttention	IO-aware attention	Optimization, Quantify	Derive memory reads: standard vs. tiled
vLLM	KV-cache paging	Diagnosis, Fluency	Calculate fragmentation at 128K context
Orca	Continuous batching	Mastery, Evaluation	Diagnose prefill starvation under load
MLPerf	System benchmarking	Quantify, Analyze	Interpret throughput vs. latency tradeoff

directly reference MLPerf benchmarking scenarios. The assessment targets differ: MLPerf evaluates hardware and software implementations; STAFFML evaluates human engineering judgment about those implementations.

These systems are not merely cited; they are the source material for STAFFML questions (Table 11). The questions test whether candidates understand the systems principles these papers discovered.

13.4 Educational Measurement

The assessment design methodology draws on three traditions. Bloom’s revised taxonomy (Anderson et al., 2001; Bloom et al., 1956) provides the six-level cognitive framework that maps to our mastery levels, validated by decades of educational research as a reliable predictor of assessment difficulty (Webb, 1997; Biggs and Collis, 1982). Evidence-Centered Design (Mislevy et al., 2003) provides the three-component assessment architecture (claim, evidence, task features) that structures our construction pipeline. Automatic Item Generation (Gierl and Haladyna, 2013) provides the theoretical foundation for template-based question generation, extended here with hardware-grounded construction and multi-model verification.

The topic taxonomy follows SKOS (W3C, 2009) for knowledge organization, faceted classification principles (Hjørland, 2013) for orthogonal axis design, and test blueprinting methodology (Millman and Greene, 1989; Case and Swanson, 2002) for coverage validation. Item Response Theory (Lord, 1980; Hambleton et al., 1991) provides the psychometric framework for future difficulty calibration once sufficient response data (≥ 30 responses per item) is collected.

The TinyTorch educational framework (Reddi, 2026d) provides a complementary pedagogical approach: where STAFFML assesses whether engineers have ML systems competencies, TinyTorch teaches

those competencies through a build-from-scratch curriculum spanning 20 modules.

14 Limitations

We identify four limitations of the current work.

No empirical validation. The bank is checked for structural coherence, physics applicability, and mathematical consistency, but it has not yet been psychometrically validated with candidate response data. Correlating zone-level performance with engineering seniority is an important next step. Until such data exist, zone and level assignments reflect the design rubric, not measured item statistics.

Static questions with chain-based progression. While individual questions are static text, the depth chain structure enables adaptive progression: a candidate who answers an L3 question correctly is presented with the L4 question on the same topic. This simulates some of the adaptivity of live interviews, though it does not replicate the free-form dialogue of whiteboard sessions. The chains (Section 3.5) approximate the probing behavior of experienced interviewers.

Architectural paradigm drift. The topic taxonomy is anchored in the current Transformer-dominant era, and identifiers such as `attention-mechanism-systems` will need revision as alternative architectures (e.g., state space models (Gu and Dao, 2024)) mature. The ikigai competency model and the five laws of Section 1 are stable under such transitions: any architecture still has an arithmetic intensity, a memory hierarchy footprint, and a communication tax, so the constants shift but the cognitive acts (diagnosing a memory wall, specifying a latency bound, evaluating a parallelism strategy) do not. As new architectures stabilize, topic labels will be deprecated and replaced

while questions targeting the same zones are re-grounded in the new constants.

Hardware vendor concentration. The corpus draws predominantly on NVIDIA hardware specifications (H100, A100, Orin), reflecting current industry adoption. Candidates with primary experience on AMD, Intel, or Google TPU hardware may find the specific constants unfamiliar, though the reasoning patterns (roofline analysis, memory budgeting, power-performance tradeoffs) are vendor-neutral. Future versions will include multi-vendor question variants. For assessment contexts, we recommend providing a hardware specification reference sheet alongside the questions, ensuring that recall of vendor-specific numbers does not gate access to the reasoning being tested.

Hardware constant maintenance. Every question is grounded in specific hardware specifications (H100: 80 GB HBM3, 3.35 TB/s, 495 TFLOPS TF32 dense; Orin: 275 TOPS INT8). These numbers are correct as of the current accelerator generation but require updates as new hardware ships. The shared infrastructure modeling framework centralizes constants to make updates tractable. Changing one constant propagates to all affected questions, and the invariant system flags any question whose napkin math references outdated specifications. The methodology is designed to be timeless even as specific numbers evolve.

15 Conclusion and Future Work

STAFFML introduces a structured approach to assessing ML systems engineering expertise. The core contribution is the *ikigai competency model*, which decomposes systems expertise into four skills (recall, analyze, design, quantify) whose pairwise intersections produce eleven cognitive zones capturing qualitatively different acts of engineering reasoning. Combined with questions that require napkin math against real hardware specifications, the model provides a vocabulary for what it means to “know” ML systems engineering at each level of professional seniority.

The four-axis classification operationalizes this model into an assessment system. The topic taxonomy organizes 87 concepts with prerequisite relationships that enable personalized study paths. The track axis grounds every question in a specific physics regime, ensuring assessment reflects real deployment constraints across cloud, edge, mobile, and TinyML.

The level axis maps to both Bloom’s cognitive taxonomy and industry engineering ladders, making the resource actionable for interview preparation and role calibration.

We plan to implement Item Response Theory (IRT) calibration once responses are collected, assigning empirical difficulty and discrimination parameters to each item and measuring the point-biserial correlation between candidate ladder levels and compound-zone performance as a construct validity check. To expand the corpus, a contribution pipeline with automated schema validation will enable industry practitioners to submit questions grounded in real production failure modes. We also envision adaptive question selection that adjusts to individual profiles, targeting the zones where a candidate is weakest. Finally, we plan to introduce incomplete-information questions that provide partial data, requiring candidates to identify what additional information they need before solving the problem. The prerequisite graph will also enable graph-guided study paths that sequence topics based on dependency structure rather than arbitrary ordering.

The corpus, schema, and quality infrastructure are available as a companion to the *Machine Learning Systems* textbook (Reddi, 2026b)⁷.

References

- Advanced Micro Devices. AMD Instinct MI300X accelerator. Data sheet, AMD, 2023. URL <https://www.amd.com/en/products/accelerators/instinct/mi300/mi300x.html>. MI300X datasheet: 192 GB HBM3, 5.3 TB/s memory bandwidth, 1,307 TFLOPS FP16 matrix.
- Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2024. Chunked prefill with stall-free batching: overlap prefill and decode to improve LLM serving tail latency.
- Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on*

⁷mlsysbook.ai/staffml

- Empirical Methods in Natural Language Processing (EMNLP)*, 2023. Grouped-query attention interpolates between multi-head and multi-query attention, shrinking KV-cache.
- Lorin W. Anderson, David R. Krathwohl, Peter W. Airasian, Kathleen A. Cruikshank, Richard E. Mayer, Paul R. Pintrich, James Rath, and Merlin C. Wittrock. *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. Longman, New York, 2001. The revised Bloom's taxonomy: Remember, Understand, Apply, Analyze, Evaluate, Create.
- AnKing Team. AnKing step deck — community quality control, 2024. URL <https://www.ankingmed.com>. Large-scale Anki deck with community-driven dedup and quality flags.
- Arm Limited. ARM Cortex-M4 processor technical reference manual. Reference manual, Arm Limited, 2020. URL <https://developer.arm.com/documentation/100166/latest>. Cortex-M4 TRM: FPU option, DSP SIMD instructions, typical deployment with 256 KB SRAM.
- John B. Biggs and Kevin F. Collis. *Evaluating the Quality of Learning: The SOLO Taxonomy (Structure of the Observed Learning Outcome)*. Academic Press, New York, 1982. SOLO taxonomy: prestructural, unistructural, multistructural, relational, extended abstract.
- Benjamin S. Bloom, Max D. Engelhart, Edward J. Furst, Walker H. Hill, and David R. Krathwohl. *Taxonomy of Educational Objectives: The Classification of Educational Goals*. David McKay Company, New York, 1956. Original Bloom's taxonomy. Revised by Anderson & Krathwohl (2001).
- Andrei Z. Broder. On the resemblance and containment of documents. In *Proceedings of the Compression and Complexity of Sequences*, pages 21–29. IEEE, 1997. doi: 10.1109/SEQUEN.1997.666900. MinHash for near-duplicate detection at web scale.
- Susan M. Case and David B. Swanson. *Constructing Written Test Questions for the Basic and Clinical Sciences*. National Board of Medical Examiners, Philadelphia, PA, 3rd edition, 2002. USMLE test blueprinting with 4+ independent classification axes per item.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. HumanEval: coding benchmark analogous to StaffML's napkin-math assessment.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022. IO-aware attention algorithm that reduces memory reads/writes from quadratic to linear in sequence length.
- Elias Frantar, Saleh Ashkboos, Torsten Hoeftler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023. Second-order post-training quantization for LLMs down to 3–4 bits.
- Mark J. Gierl and Thomas M. Haladyna. *Automatic Item Generation: Theory and Practice*. Routledge, New York, 2013. doi: 10.4324/9780203803912. Foundational text on template-based assessment item generation.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *Conference on Language Modeling (COLM)*, 2024. Selective state space model with linear-time sequence scaling; alternative to attention-based architectures.
- Ronald K. Hambleton, Hariharan Swaminathan, and H. Jane Rogers. *Fundamentals of Item Response Theory*. Sage Publications, Newbury Park, CA, 1991. Practical guide to IRT. 30+ responses needed for stable calibration.
- Birger Hjørland. Facet analysis: The logical approach to knowledge organization. *Information Processing & Management*, 49(2):545–557, 2013. Faceted classification: independent orthogonal axes rather than a single hierarchy.
- Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Xu Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. GPipe: Efficient training of giant neural networks using pipeline parallelism. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019. Pipeline parallelism for training models that exceed single-device memory.

- Chip Huyen. *Designing Machine Learning Systems*. O'Reilly Media, Sebastopol, CA, 2022. ISBN 978-1098107963. ML systems design textbook focused on production deployment.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*. ACM, 2023. doi: 10.1145/3600006.3613165. vLLM: virtual memory paging for KV-cache reduces fragmentation and enables higher throughput.
- LeetCode Inc. LeetCode — online coding platform, 2024. URL <https://leetcode.com>. 2,800+ validated coding problems with community-driven difficulty calibration.
- Ji Lin, Wei-Ming Chen, Yujun Lin, John Cohn, Chuang Gan, and Song Han. MCUNet: Tiny deep learning on IoT devices. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024. Activation-aware weight-only quantization preserving salient channels. MLSys 2024 best paper.
- Frederic M. Lord. *Applications of Item Response Theory to Practical Testing Problems*. Lawrence Erlbaum Associates, Hillsdale, NJ, 1980. Foundational IRT text. b-parameter (difficulty), a-parameter (discrimination).
- Peter Mattson, Christine Cheng, Cody Coleman, Greg Diamos, Paulius Micikevicius, David Patterson, Hanlin Tang, Gu-Yeon Wei, Peter Bailis, Victor Bittorf, David Brooks, Dehao Chen, Debojyoti Dutta, Udit Gupta, Kim Hazelwood, Andrew Hock, Xinyuan Huang, Atsushi Ike, Bill Jia, Daniel Kang, David Kanter, Naveen Kumar, Jeffery Liao, Guokai Ma, Deepak Narayanan, Tayo Oguntebi, Gennady Pekhimenko, Lillian Pentecost, Vijay Janapa Reddi, Taylor Robie, Tom St John, Carole-Jean Wu, Lingjie Xu, Cliff Young, and Matei Zaharia. MLPerf Training Benchmark. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 2, pages 336–349, 2020. The original MLPerf Training benchmark establishing standardized ML system measurement.
- Samuel Messick. Validity of psychological assessment: Validation of inferences from persons' responses and performances as scientific inquiry into score meaning. *American Psychologist*, 50(9):741–749, 1995. doi: 10.1037/0003-066X.50.9.741. Unified validity framework: content, construct, consequential validity.
- Jason Millman and Jennifer Greene. Test blueprints. In Robert L. Linn, editor, *Educational Measurement*. American Council on Education / Macmillan, New York, 3rd edition, 1989. Test blueprint / table of specifications for coverage validation.
- Robert J. Mislevy, Russell G. Almond, and Janice F. Lukas. A brief introduction to evidence-centered design. Technical Report RR-03-16, Educational Testing Service, Princeton, NJ, 2003. ECD framework: claims, evidence, task features for assessment design.
- NeetCode. NeetCode 150 — curated coding interview problems, 2024. URL <https://neetcode.io/practice>. 150 problems organized by pattern with difficulty progression.
- NVIDIA Corporation. NVIDIA H100 tensor core GPU architecture. White paper, NVIDIA Corporation, 2022a. URL <https://resources.nvidia.com/en-us-tensor-core/gtc22-whitepaper-hopper>. Hopper architecture whitepaper: 80 GB HBM3, 3.35 TB/s, 989 TFLOPS FP16 dense tensor-core throughput.
- NVIDIA Corporation. NVIDIA Jetson AGX Orin series technical brief. Technical brief, NVIDIA Corporation, 2022b. URL <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/>. Jetson Orin datasheet: 275 TOPS INT8 within a 15–60 W configurable power envelope.
- OpenAI. Gpt-4 technical report. Technical report, 2023.

- Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. IEEE, 2020. doi: 10.1109/SC41405.2020.00024. ZeRO optimizer partitioning eliminates memory redundancy in data parallelism.
- Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 3505–3506. ACM, 2020. doi: 10.1145/3394486.3406703. Training system combining ZeRO, pipeline parallelism, and mixed precision for 100B+ parameter models.
- Vijay Janapa Reddi. CS249r: Tiny machine learning — harvard university, 2024. URL <https://sites.google.com/g.harvard.edu/cs249-tinyml-2024>. Graduate course on ML systems spanning cloud to embedded deployment.
- Vijay Janapa Reddi. *Machine Learning Systems at Scale*. MIT Press, 2026a. URL <https://mlsysbook.ai>. Volume II of the Machine Learning Systems textbook. Forthcoming.
- Vijay Janapa Reddi. *Machine Learning Systems*. MIT Press, 2026b. URL <https://mlsysbook.ai>. Two-volume textbook on ML systems following the Hennessy & Patterson pedagogical model. Volume I covers single-machine systems; Volume II covers distributed systems at scale.
- Vijay Janapa Reddi. MLSysIM: A physics simulator for machine learning systems. <https://mlsysbook.ai/mlsysim>, 2026c. Companion simulator for the ML Systems textbook. Provides a shared physics engine with hardware constants, roofline analysis, and quantitative verification of systems reasoning.
- Vijay Janapa Reddi. TinyTorch: Building machine learning systems from first principles. *arXiv preprint arXiv:2601.19107*, 2026d. URL <https://arxiv.org/abs/2601.19107>. Educational ML framework with 20 modules teaching ML as systems engineering from first principles. Companion to the Machine Learning Systems textbook.
- Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, Ramesh Chukka, Cody Coleman, Sam Davis, Pan Deng, Greg Diamos, Jared Duke, Dave Fick, J. Scott Gardner, Itay Hubara, Sachin Idgunji, Thomas B. Jablin, Jeff Jiao, Tom St. John, Pankaj Kanwar, David Lee, Jeffery Liao, Anton Lokhmotov, Francisco Massa, Peng Meng, Paulius Micikevicius, Colin Osborne, Gennady Pekhimenko, Arun Tejusve Raghunath Rajan, Dilip Sequeira, Ashish Sirasao, Fei Sun, Hanlin Tang, Michael Thomson, Frank Wei, Ephrem Wu, Lingjie Xu, Koichi Yamada, Bing Yu, George Yuan, Aaron Zhong, Peizhao Zhang, and Yuchen Zhou. MLPerf Inference Benchmark. In *Proceedings of the ACM/IEEE International Symposium on Computer Architecture (ISCA)*, pages 446–459. IEEE, 2020. doi: 10.1109/ISCA45697.2020.00045. Standardized benchmarks for ML system performance across inference scenarios.
- Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992. Association for Computational Linguistics, 2019. doi: 10.18653/v1/D19-1410. Siamese BERT for fast semantic similarity; used in StaffML deduplication.
- Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019. Tensor parallelism for training large language models across GPUs.
- Dagobert Soergel. *Organizing Information: Principles of Data Base and Retrieval Systems*. Academic Press, Orlando, FL, 1985. User warrant vs literary warrant in knowledge organization.
- Yangshun Tay. Grind 75 — dynamically ranked interview problems, 2024. URL <https://www.techinterviewhandbook.org/grind75>. 169 problems ranked by frequency, pattern coverage, and difficulty.
- Martin Thompson. Mechanical sympathy, 2011. URL <https://mechanical-sympathy.blogspot.com/>. Blog. The term

- describes software that works with the hardware rather than against it, inspired by racing driver Jackie Stewart’s philosophy.
- W3C. SKOS simple knowledge organization system reference. W3c recommendation, World Wide Web Consortium, 2009. URL <https://www.w3.org/TR/skos-reference/>. Standard for broader/narrower/related concept relationships.
- Norman L. Webb. Criteria for alignment of expectations and assessments in mathematics and science education. Technical Report Research Monograph No. 6, National Institute for Science Education, University of Wisconsin–Madison, 1997. Depth of Knowledge (DOK) framework: four levels of cognitive complexity for assessment alignment.
- Grant Wiggins and Jay McTighe. *Understanding by Design*. Association for Supervision and Curriculum Development, Alexandria, VA, 2nd edition, 2005. Backward design methodology: desired results, acceptable evidence, then learning plan.
- Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785. Canonical roofline performance model: arithmetic intensity vs. memory bandwidth vs. peak compute.
- Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 521–538. USENIX Association, 2022. Iteration-level scheduling (continuous batching) for transformer serving, eliminating padding waste.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and chatbot arena. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.